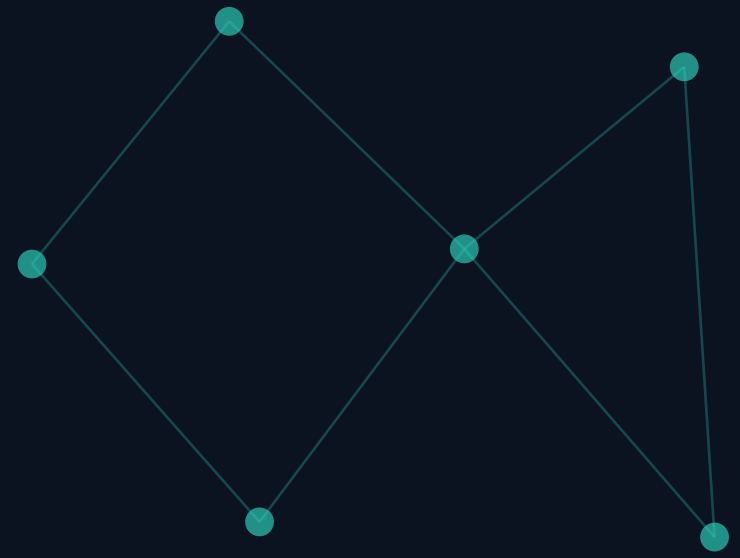
CLASE 9 · UNIDAD 4 · SEMANA 9

Memoria, contexto y grafos

El modelo no recuerda nada de una llamada a la otra: todo lo que llamamos memoria es decidir qué vuelve a entrar a la ventana.



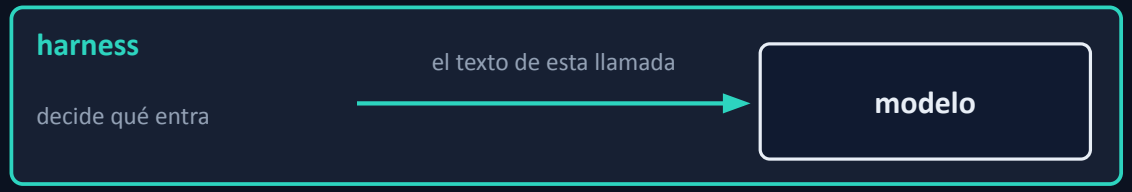
APERTURA · LO QUE HACE EL HARNESS

Claude se acuerda. ¿Quién se acuerda?

Lo que se ve en Claude. Una pregunta que dice “¿y eso?” se entiende: Claude sigue el hilo. Un chat muy largo sigue andando, aunque gasta más rápido el límite de uso del plan; y cuando la conversación ya casi no entra en lo que el modelo puede leer de una vez —su ventana—, Claude resume la parte vieja y sigue (en los planes pagos). Y en un chat nuevo se acuerda de lo que se le contó otro día.

El modelo no hace nada de eso. Es una función: recibe un texto, devuelve el próximo token y no guarda nada de una llamada a la otra. Todo lo de arriba lo hace el programa que lo rodea, que en la industria se llama harness (arnés): decide qué texto le llega al modelo en cada llamada. Anthropic lo explica así: cada mensaje vuelve a mandar la conversación entera.

Con la API, el harness lo escribimos nosotros. La API de mensajes no guarda estado: en cada llamada hay que mandarle la conversación completa. Al construir un sistema propio —el segundo proyecto, los agentes de la unidad que viene—, cada cosa que Claude hace por nosotros hay que hacerla a mano. Esta clase abre el harness, de a una pieza por vez.



lo que se ve en Claude	lo que hace el harness	dónde lo armamos
sigue el hilo	re-envía la conversación entera en cada mensaje	1 · la siguiente pregunta
los chats largos gastan más	cada mensaje vuelve a mandar todo lo anterior	2 · la conversación larga
resume lo viejo	un resumen ocupa el lugar de los turnos viejos	3 · cuando ya no entra
se acuerda en un chat nuevo	escribe memorias afuera y las vuelve a traer	4 · al día siguiente

el cuarto se arma la semana que viene

El modelo no recuerda nada: lo que recuerda es el programa que lo rodea. Hoy se abre ese programa, el harness, y se arma a mano lo que Claude hace por nosotros.

Del modelo por dentro a lo que lo rodea

Hasta acá, el modelo por dentro. Lo armamos desde abajo: el bigrama, la atención, la tokenización, los embeddings, el entrenamiento y el alineamiento. Todo eso termina en un objeto fijo: unos pesos que, dado un texto, predicen el próximo token.

Después, lo que se le da para leer. Con prompt engineering aprendimos a escribir una llamada; con RAG, a ponerle delante una fuente que no estaba en los pesos. Cada clase agregó algo a la entrada, sin tocar el modelo.

Hoy, muchas llamadas seguidas. Una conversación son muchas llamadas al mismo modelo, y en cada una alguien decide qué entra: lo que se dijo antes, lo que se recuperó, lo que se sabe del usuario. Ese alguien es el harness. El paso que sigue en el curso es que esas decisiones las tome el propio modelo, con herramientas: eso es un agente.



cada escalón agrega algo a la entrada; el modelo es el mismo

En toda la clase no cambia un solo peso del modelo. Cambia qué se le manda en cada llamada, y quién lo decide.

La conversación que vamos a seguir

Un usuario, una conversación. Toda la clase sigue a un mismo usuario: trabaja en una pesquera de Vigo y le pregunta por normativa europea al RAG de la clase pasada, sobre el mismo corpus. Esta semana es una sola conversación de 21 turnos, del 0 al 20; en cada turno, el usuario escribe y el sistema contesta.

Dos datos plantados. En el turno 0 el usuario cuenta dónde trabaja y cómo quiere las respuestas; en el turno 7, qué artículo va a citar en un informe. Se dicen una sola vez, temprano. Después del turno 20 los vuelve a pedir, y lo que se mide es siempre lo mismo: ¿siguen en la ventana cuando hacen falta?

Por qué alcanzan veinte turnos. La ventana de nuestra placa es chica: 4.096 tokens. Las de Claude son decenas de veces más grandes: los mismos problemas llegan mucho más tarde, pero llegan.

0	Antes de empezar, dos cosas: trabajo en una pesquera de **Vigo**, y necesito las respuestas siempre con el artículo citado.
1	¿Qué reglamento fijó las posibilidades de pesca del Mar Báltico para 2012? *sistema: «El Reglamento (UE) nº 1256/2011 fijó las posibilidades de pesca del Mar Báltico para 2012.»*
2	¿Y qué dice sobre el bacalao? **← la siguiente pregunta**
3	¿Ese reglamento cambió algo de la norma del año anterior?
4	¿Y para 2013?
5	Pasando al Mar del Norte: ¿cuándo se puede cerrar una pesquería de un día para otro?
6	¿A qué especies alcanza esa medida?
7	Anotá esto para el informe: el que voy a citar es el Reglamento (UE) nº 1256/2011, **artículo 9**.
8–20	*Trece preguntas sueltas: aviones, patentes, colesterol, aduanas, biocidas, una deuda búlgara, algodón transgénico, radiadores chinos, ruido de trenes, aceite de oliva, metrología, un dulce checo, un programa de justicia.*
después del 20	«¿En qué ciudad te dije que trabajo?» · «¿Qué reglamento y qué artículo te dije que iba a citar?»

Una conversación de 21 turnos con dos datos plantados: Vigo y el artículo 9. Toda la semana se pregunta lo mismo: ¿siguen en la ventana cuando hacen falta? ▸ Notebook §1 — el guion de la conversación.

Seis momentos en la vida de la conversación

La clase sigue la conversación mientras crece. En cada momento de su vida se rompe algo distinto, y cada arreglo deja el problema del momento que sigue. El pie de cada lámina dice en qué momento estamos.

Esta semana, los tres primeros.

1 · La siguiente pregunta: desde el turno 2, cada pregunta depende de las anteriores; "¿Y qué dice sobre el bacalao?" no se entiende sin el turno 1.

2 · La conversación larga: a medida que se acerca al turno 20, cada turno re-envía todo lo anterior; cuesta cada vez más, y la ventana se acaba.

3 · Cuando ya no entra: después del turno 20 algo hay que sacar de la ventana; ¿se van Vigo y el artículo 9?

La semana que viene, los otros tres.

4 · Al día siguiente: otra sesión, y de la conversación de ayer no queda nada.

5 · Datos conectados: una pregunta que une datos de varias sesiones, y que la similitud no conecta.

6 · ¿Quién decide?: hasta ahí decide el código, con reglas; el último paso es que decida el modelo.



cada momento, en el punto de la conversación donde algo se rompe

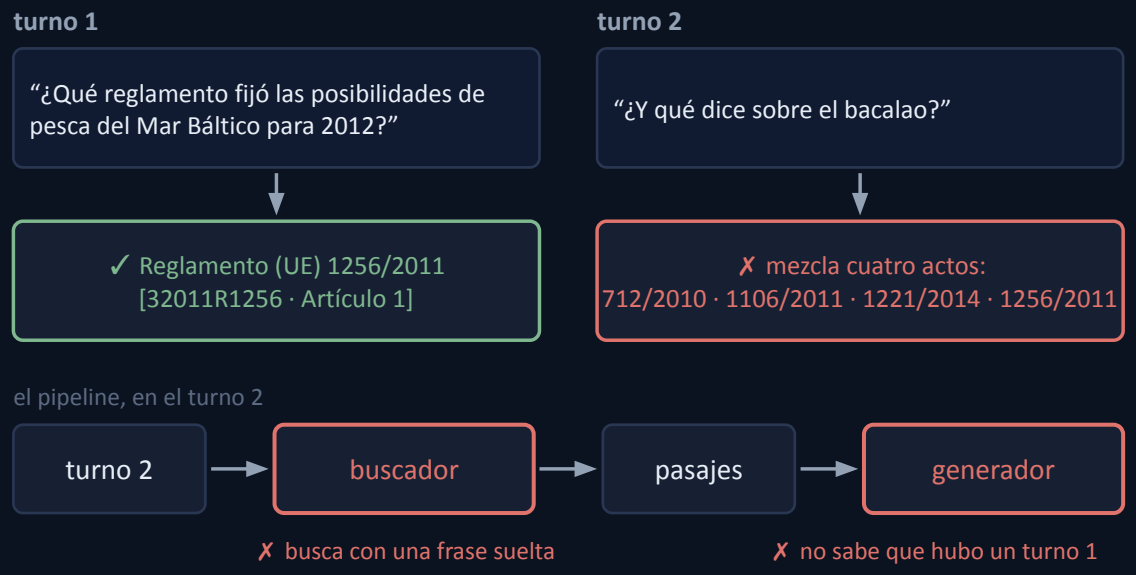
Cada momento es un punto en la vida de la conversación donde algo se rompe. Y cada arreglo deja el problema del momento que sigue.

La siguiente pregunta

En Claude, la siguiente pregunta anda. Se pregunta algo, después “¿y eso?”, y Claude entiende. El pipeline que armamos en la clase de RAG, en cambio, recibe una pregunta, recupera, responde con cita y se olvida de todo: cada pregunta es la primera. Hoy, la siguiente.

El turno 1 sale bien. “¿Qué reglamento fijó las posibilidades de pesca del Mar Báltico para 2012?” El sistema recupera el reglamento y responde con su número y el artículo.

El turno 2 falla, y falla dos veces. “¿Y qué dice sobre el bacalao?” El generador recibe una frase corta sin saber de qué se venía hablando: no hay mar, ni año, ni reglamento, ni siquiera sobre qué norma “dice”. Y el buscador busca con esas mismas palabras, así que trae bacalao de cualquier lado. Son dos fallas distintas, porque el mismo arreglo no sirve para las dos: darle la conversación al generador lo arregla; dársela al buscador, no. Las arreglamos en ese orden: primero que el modelo sepa qué se dijo, después que el buscador sepa qué buscar.



dos fallas distintas: primero que recuerde, después que busque bien

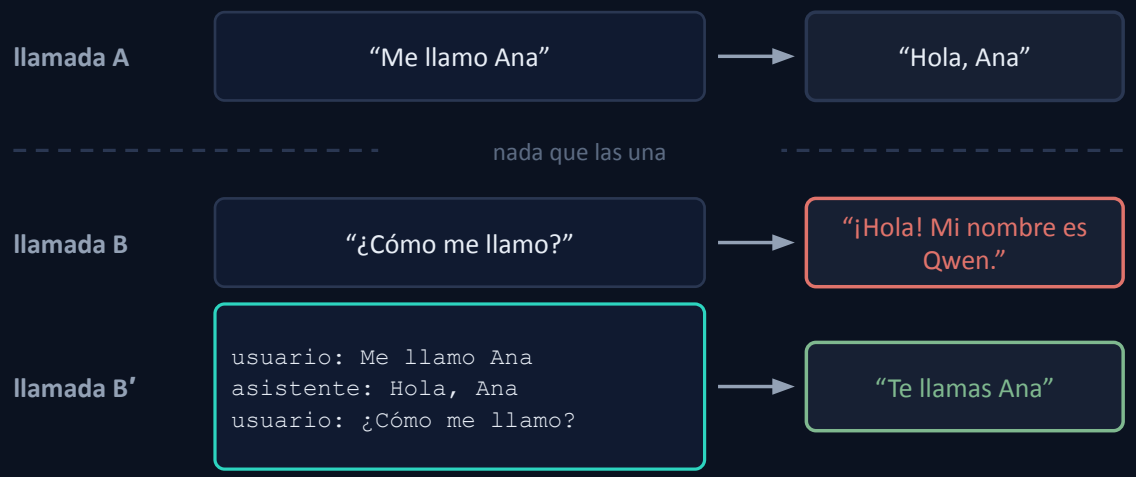
Un RAG responde preguntas; una conversación es una secuencia de preguntas que dependen unas de otras. Lo que falta no es recuperar mejor: es recordar. ▸ Notebook §1 — el pipeline de RAG, dos turnos seguidos.

El modelo no recuerda: lee

Por qué el modelo no sabe del turno 1. Cada llamada es una función de su entrada: entra un texto, sale una distribución sobre el próximo token. Los pesos están congelados desde el entrenamiento —como vimos al arrancar RAG— y nada más persiste. Lo que no tiene delante en esta llamada, para el modelo no existe.

Cada llamada a la API también empieza de cero. OpenAI lo dice sin vueltas: “each text generation request is independent and stateless”; Anthropic pide mandar el historial completo en cada llamada. Para que haya conversación, alguien tiene que guardar la lista de mensajes y mandarla entera en cada llamada: el harness —el servidor del chat, un framework, nuestro script—.

Guardar la conversación no es recordarla. Algunas APIs ofrecen guardar la lista de su lado, para no re-enviarla. Eso cambia quién la guarda, no lo que hace el modelo: en cada llamada todos los turnos vuelven a entrar a la ventana, y por eso se cobran otra vez como entrada. Como el protagonista de Memento, el modelo no forma recuerdos nuevos: sabe sólo lo que tiene escrito delante.



la memoria es la lista, y la lista la tiene el harness

Nota de laboratorio: la API de chat de Ollama funciona igual — recibe la conversación entera en cada llamada.

El modelo no tiene estado entre llamadas. Lo que parece memoria es el harness mandando la conversación entera otra vez. ▸ Notebook §1 — las dos llamadas, sin y con la lista.

Un chat es una lista que crece

Una lista de mensajes con roles. Lo que el harness manda es una lista: un mensaje de sistema con las instrucciones y después turnos alternados de usuario y de asistente. Pero el modelo no lee listas. La plantilla de chat (chat template) la convierte en una sola secuencia de tokens, con tokens especiales que marcan dónde empieza y termina cada turno y de quién es: los mismos que vimos en tokenización.

En cada turno, la lista crece y se re-envía entera. El harness agrega la pregunta nueva, llama al modelo, agrega la respuesta, y en el turno siguiente vuelve a mandar todo. El turno n lleva adentro los n − 1 intercambios anteriores.

La consecuencia que ordena la clase. Si lo que el modelo "recuerda" es lo que está en esa lista, editar la lista es editar la memoria: sacar turnos, resumirlos, traer cosas de afuera. Toda la clase trata de cómo editarla, y de quién lo hace.



la lista crece en cada turno; la plantilla la aplana en una secuencia de tokens

Cada turno se paga entero de nuevo. Y lo que no esté en esa lista, para el modelo no pasó. ▸ Notebook §1 — la plantilla de chat, renderizada para dos turnos, y el prompt creciendo turno a turno.

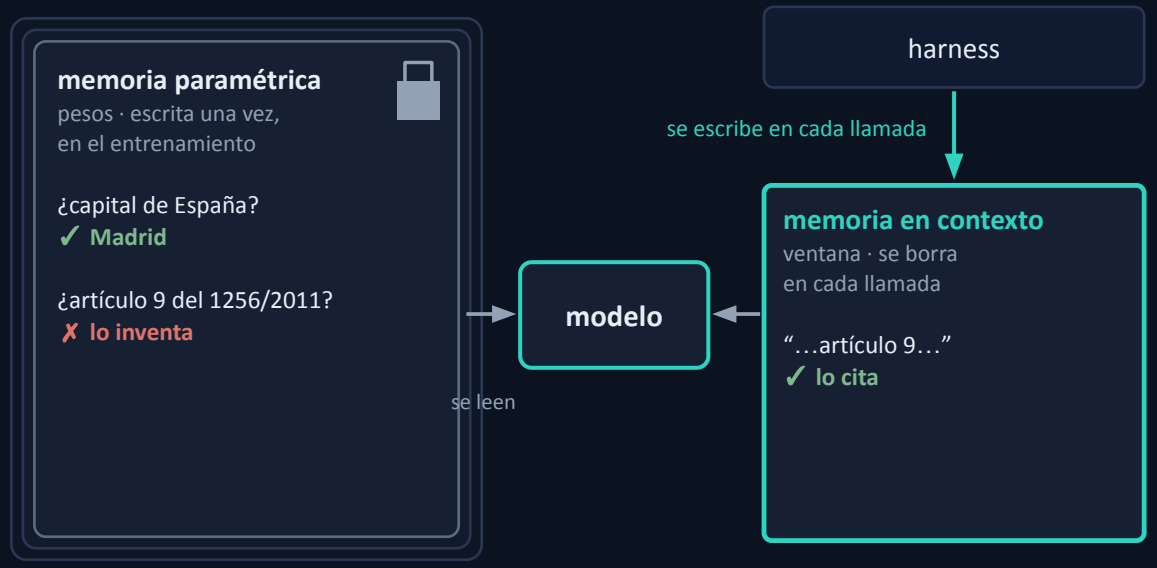
Las dos memorias del modelo

Si no guarda nada entre llamadas, ¿de dónde sale lo que sí sabe? De dos lugares distintos, que la literatura nombra: la memoria paramétrica, en los pesos, y la memoria en contexto, en la ventana.

Memoria paramétrica: los pesos. Se escriben una vez, en el entrenamiento. No guardan el texto leído: guardan regularidades, lo que se repitió muchas veces. Es una compresión con pérdida, un recuerdo difuso. Por eso el modelo sabe lo frecuente —"Madrid es la capital de España" lo leyó miles de veces— y no lo raro: "el artículo 9 del Reglamento 1256/2011" —el que el usuario anota en el turno 7— lo vio, con suerte, una vez. Cuando se le pregunta lo raro, lo inventa con la misma fluidez. Por eso existe RAG.

Memoria en contexto: la ventana. Lo que está en el prompt está entero, token por token, y el modelo lo puede citar tal cual: si el artículo 9 está en la ventana, no hace falta recordarlo. Pero dura una llamada: al terminar, se libera. Es la memoria de trabajo del modelo.

El hilo con Transformers. "Darle memoria al modelo" fue la atención: cada token nuevo mira a todos los anteriores y hace un promedio ponderado de lo que encuentra. Es memoria en contexto, y llega hasta el borde de la ventana, ni un token más allá.



todo lo de hoy pasa en la ventana

Una memoria enorme que no se puede escribir, y una exacta que dura una llamada. Todo lo que hoy llamamos memoria es administrar la segunda. ▸ Notebook §1 — lo frecuente, lo raro y lo raro con la fuente delante.

El mapa: una computadora

Una computadora tiene tres memorias; el modelo trae dos. La RAM es lo que el procesador usa ahora: rápida, chica, y se borra al apagar. El disco guarda los archivos: grande y permanente, pero el procesador no trabaja sobre el disco; para usar un archivo, primero hay que subirlo a la RAM. Y la memoria de fábrica sólo se lee. Las dos memorias del modelo son la RAM —la ventana— y la de fábrica —los pesos—. El disco lo pone el sistema: son los almacenes —un archivo, una base de datos, un índice de vectores como el de RAG—, y lo que el modelo necesite de ahí, alguien lo tiene que subir a la ventana.

El harness es el sistema operativo. Decide qué sube a la ventana, qué se saca y qué se guarda afuera. Un sistema de investigación, MemGPT, tomó esta analogía al pie de la letra.

Lo que ya sabemos hacer: subir. RAG, visto desde acá, es una flecha del disco a la RAM: elegir qué pasaje sube a la ventana, desde un disco que escribió otro. La industria nombra cuatro de estas flechas —seleccionar (select), comprimir (compress), escribir (write) y aislar (isolate)— y las vamos a ir encendiendo a medida que cada problema las pida. Por ahora, sólo la primera está prendida.



una flecha encendida; las otras tres, a medida que cada problema las pida

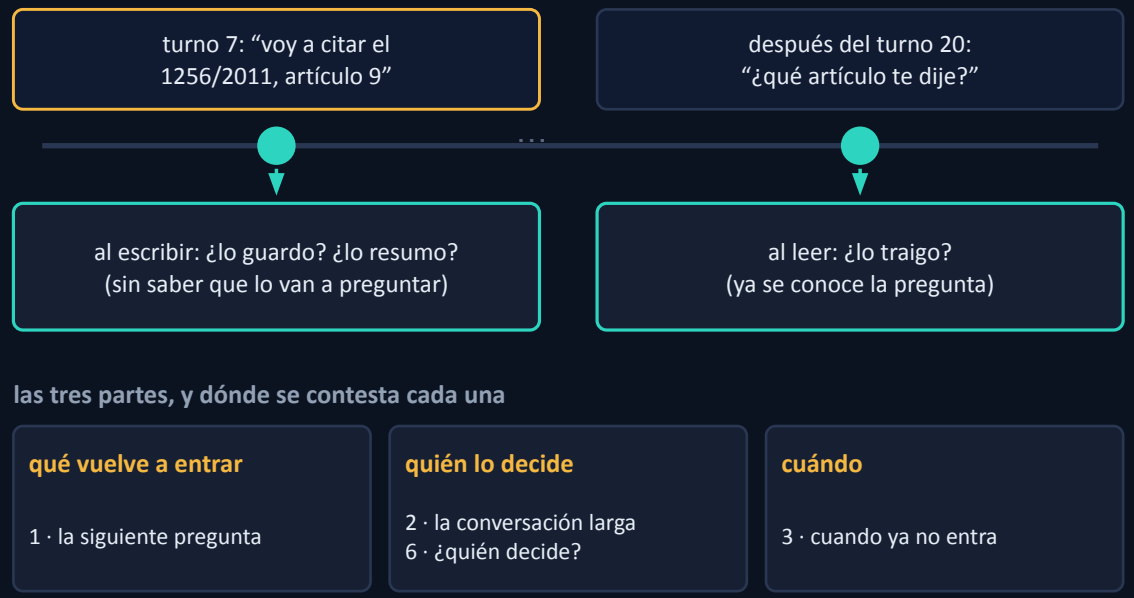
Un sistema con memoria es un sistema operativo para el modelo: decide qué sube a la ventana, qué se saca y qué se guarda afuera. Hasta acá sólo sabemos subir; lo demás se arma en esta clase.

La pregunta madre

Recordar es decidir qué vuelve a entrar. Si el modelo sólo sabe lo que está en la ventana, en cada llamada algo de lo que pasó vuelve a entrar y algo no. La pregunta que atraviesa la clase tiene tres partes: qué vuelve a entrar, quién lo decide y cuándo.

Con los datos plantados. En el turno 7 el usuario dice qué artículo va a citar; después del turno 20 pregunta "¿qué artículo te dije?". Para contestar, el artículo 9 tiene que estar en la ventana. Escribir y leer son del sistema: escribir es guardar en su memoria; leer, traer de ahí a la ventana. Se puede decidir al escribir, cuando el dato llega —guardarlo, resumirlo—: sale barato después, pero sin saber qué se va a preguntar. O guardar todo tal cual y decidir al leer, cuando llega la pregunta: se sabe qué importa, pero se paga en cada pregunta y depende de que la búsqueda acierte. Ya pasaba en RAG: el embedder decide al indexar; el re-puntuador, al leer.

Una parte por momento. Qué vuelve a entrar se contesta en el momento 1, la siguiente pregunta; quién lo decide, en el 2, la conversación larga, y otra vez en el 6, cuando decide el modelo; cuándo, en el 3, cuando ya no entra. Y una regla para todo lo que sigue: mientras todo entre en la ventana, se manda todo —si entra, se pega—. Cada técnica tiene que ganarle a eso.



La pregunta madre: qué vuelve a entrar a la ventana, quién lo decide y cuándo. Cada momento de la conversación contesta una parte.

El buscador no vio el turno 1

Volvamos al turno 2: falta el buscador. Con el historial adentro, el generador ya entiende "¿y qué dice sobre el bacalao?": ve el turno 1 y sabe que se habla del Báltico y de 2012. Falta el otro lado del pipeline.

El buscador sigue viendo sólo el último turno. Embebe "¿Y qué dice sobre el bacalao?" y busca. Sin mar, sin año y sin reglamento, el vector cae en cualquier pasaje que hable de bacalao. El correcto entra cuarto, entre modificaciones de otros reglamentos del bacalao y las posibilidades de pesca de 2015: acierta de casualidad. En los dos turnos que siguen, el 3 ("¿ese reglamento cambió algo?") y el 4 ("¿y para 2013?"), ya no entra.

Y la respuesta sale fluida igual. El generador entiende la pregunta y la contesta con lo que le llegó, mezclando actos. En RAG separamos dos lugares donde un sistema falla, recuperar y generar: acá falla recuperar, y no por un mal embedder, sino porque lo que recibe ya no es una pregunta que se sostenga sola.



una sola de las cinco es del reglamento correcto

En una conversación, el turno del usuario deja de ser una consulta de búsqueda. La mitad de su significado está en turnos que el buscador nunca vio. ▸ Notebook §2 — el top-5 de "¿y qué dice sobre el bacalao?", buscado solo.

Tres formas de depender, y un cambio de tema

No todas las preguntas dependen igual. Una pregunta que no se sostiene sola (non-standalone) tiene la mitad de su significado en otro turno, y esa mitad puede faltar de tres formas. Hay además un cuarto caso, que parece dependiente y no lo es.

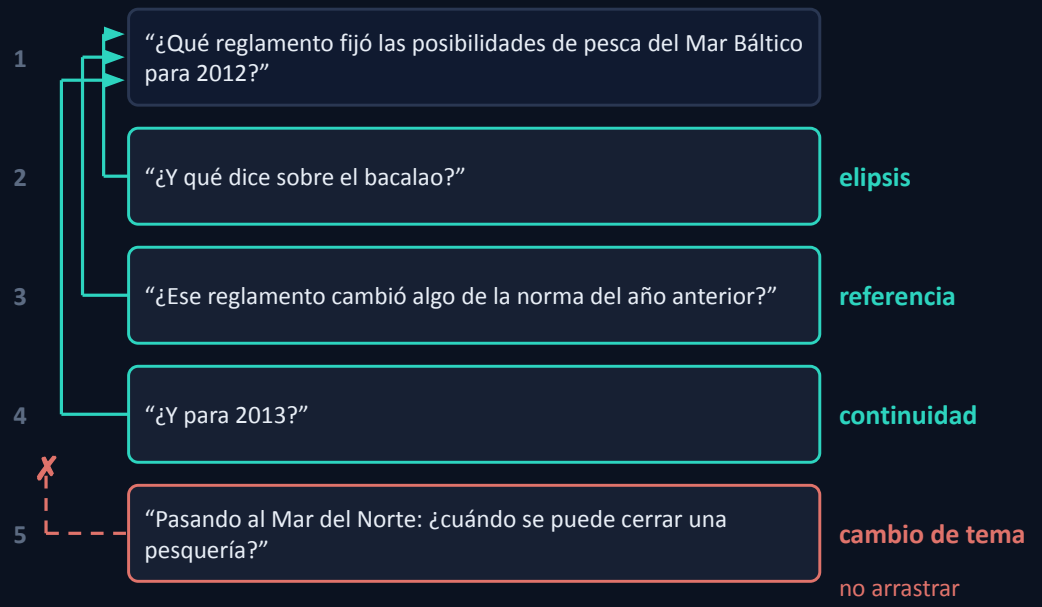
Elipsis: se omite lo que ya se dijo.

Referencia: se nombra algo de antes con otra palabra.

Continuidad: se mantiene casi todo y cambia una cosa.

Cambio de tema: parece una continuación, pero no tiene que arrastrar nada de lo anterior.

Por qué conviene nombrarlas. Cada tipo falla distinto, y lo que arregla uno puede romper otro: arrastrar contexto ayuda a las tres formas de depender y arruina el cambio de tema. Un benchmark de RAG conversacional, MTRAG (110 conversaciones, 842 tareas), encontró que los turnos tardíos y las preguntas que no se sostienen solas están entre lo más difícil para los sistemas actuales.



lo que arregla una dependencia puede romper otra

Una pregunta dependiente se entiende con la conversación, y se busca sin ella. Antes de arreglarla conviene saber de qué forma depende.

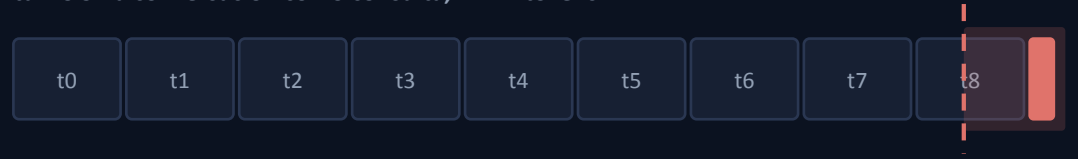
Buscar con toda la conversación

El arreglo obvio: que el buscador vea todo. Si al buscador le falta contexto, se le concatena la conversación entera. En MTRAG, un benchmark grande, pierde contra el último turno solo y trae los mismos pasajes una y otra vez. En nuestra conversación, que es corta, gana por poco —3 de 5 contra 2 de 5—, pero ya muestra los síntomas: repite pasajes y falla el cambio de tema.

Tres razones, todas conocidas. Primero, un solo vector promedia toda la conversación: la pregunta de ahora pesa una fracción y el vector casi no se mueve —como el pedazo de tres temas de RAG—; de ahí los pasajes repetidos. Segundo, el embedder lee hasta 1.024 tokens y corta lo que sigue: en orden cronológico, se pierde el turno más nuevo, el que dice qué se pregunta. En el turno 9 el usuario pregunta por patentes y el buscador trae el Báltico del principio. Tercero, BM25 no tiene ese límite: los dos canales de la búsqueda híbrida buscan textos distintos.

La segunda razón tiene parche. Poner el turno nuevo primero. La primera no: un solo vector para toda la conversación siempre la promedia.

turno 9: la conversación como consulta, 1.121 tokens



máximo del embedder: 1.024 tokens

se trunca el turno que importa

turnos 9, 10 y 11: los mismos tres actos del Báltico del principio

turno 9 · patentes	Báltico 2013 · Báltico 2012 · Báltico 2014
turno 10 · colesterol	Báltico 2013 · Báltico 2012 · Báltico 2014
turno 11 · aduanas	Báltico 2013 · Báltico 2012 · Báltico 2014

con el turno nuevo primero, el 9 entra entero y trae lo mismo: el promedio pesa más

turnos 2 a 6: con la conversación entera, 1,8 actos repetidos por turno; con el último, ninguno

Más conversación en la consulta no es más contexto. Es un promedio que no se mueve, y si la conversación es larga, la pregunta de ahora queda afuera. ▸ Notebook §2 — los resultados repetidos; §3 — dónde corta el embedder.

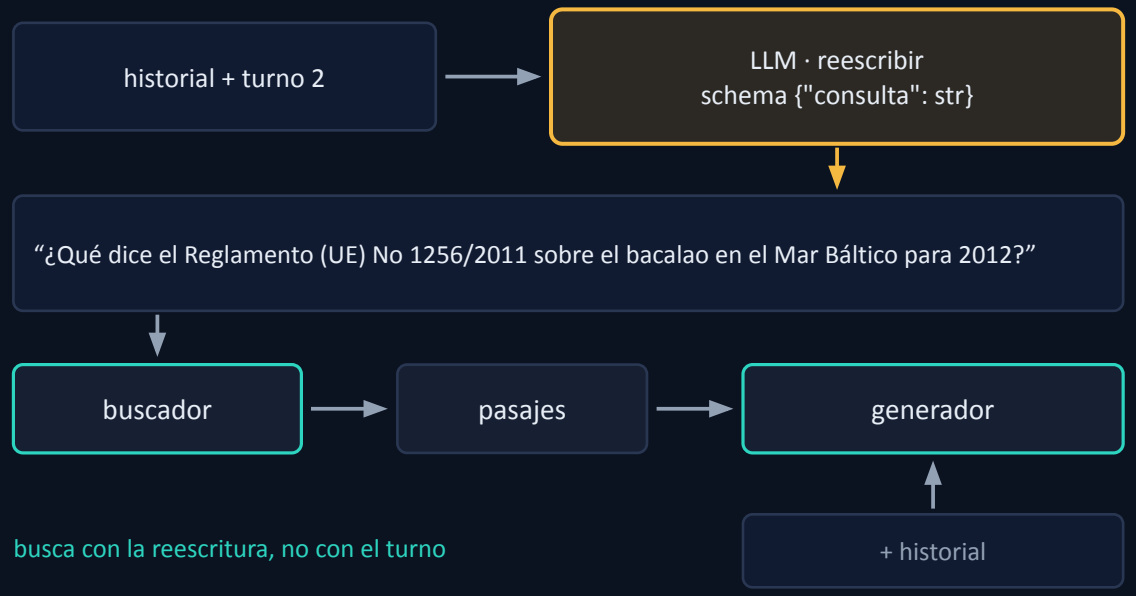
Reescribir la pregunta

Antes de buscar, completar la pregunta. Una llamada al modelo, con los últimos turnos delante, reescribe el turno del usuario como una pregunta que se sostiene sola: “¿Y qué dice sobre el bacalao?” pasa a ser “¿Qué dice el Reglamento (UE) No 1256/2011 sobre el bacalao en el Mar Báltico para 2012?”. El número lo sacó de la respuesta del turno 1. Con eso se busca. Es la reescritura de la consulta (query rewriting, o condense question).

Está medido. Reescribir y después responder funciona: el reescritor se acerca a una reescritura humana (Vakulenko et al., WSDM 2021). En MTRAG, reescribir le ganó al último turno solo en todas las métricas y con todos los modelos.

Con un schema de un solo campo. La salida se restringe a {"consulta": str}, con lo que vimos de structured outputs: no hay lugar para razonar ni para contestar: sólo para una consulta. Ocupa el lugar de HyDE, la llamada que en RAG escribía una respuesta hipotética para buscar con ella.

Desde acá, cada pieza recibe lo suyo. El buscador recibe la reescritura; el generador, el historial y los pasajes.



una llamada más por turno, en el lugar que ocupaba HyDE

El buscador no necesita la conversación: necesita la pregunta que la conversación implica, escrita entera. ▸ Notebook §2 — la reescritura y su top-5, contra el turno solo.

Cómo se equivoca un reescritor

El reescritor es un modelo, y falla como un modelo. Lo grave es que falla sin avisar: la consulta sale prolija, bien escrita, y busca otra cosa.

Cuatro fallas típicas.

Eco: devuelve el turno casi igual, y no se ganó nada.

Responder: la consulta trae metida una conjetura del modelo y la búsqueda sale dirigida por ella, como con HyDE pero sin quererlo.

Inventar precisiones: agrega un año o un número de reglamento que nadie nombró, y acota la búsqueda al lugar equivocado.

Arrastrar el tema viejo: el usuario cambió de tema y la reescritura sigue en el anterior.

Cada una rompe una propiedad. Ye et al. (EMNLP 2023) nombran cuatro: corrección, claridad, informatividad y no redundancia. El eco falla en informatividad; responder e inventar, en corrección; arrastrar el tema, en corrección y claridad.

La primera defensa es el prompt. El del reescritor del notebook nombra las fallas para evitarlas, y en nuestra corrida no cayó en ninguna: las tarjetas son ejemplos armados.

✗ **eco** informatividad	“¿Y qué dice sobre el bacalao?” — vuelve casi igual
✗ **responde** corrección	“¿…para 2012? El reglamento le dio más días de pesca”
✗ **inventa** corrección	“¿…el reglamento del Báltico de 2013?” — se habló de 2012
✗ **arrastra el tema** corrección y claridad	“¿Cuándo se puede cerrar una pesquería del Báltico en 2012?” — para “Pasando al Mar del Norte…”
✓ **buena** las cuatro propiedades	“¿Qué dice el Reglamento (UE) No 1256/2011 sobre el bacalao en el Mar Báltico para 2012?”

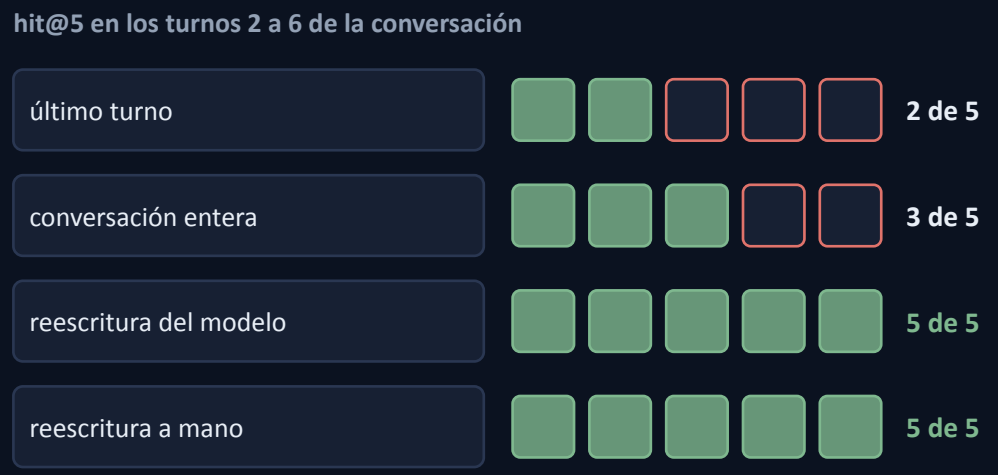
las cinco se leen bien; sólo una busca lo que se preguntó

Un reescritor falla sin avisar: la consulta sale prolija y busca otra cosa. Las reescrituras se leen, no sólo se miden.

Una reescritura se juzga por lo que trae

No se juzga leyéndola. Una reescritura puede leerse impecable y traer el pasaje equivocado, o leerse rara y traer el correcto: se juzga por la recuperación que produce. Ye et al. lo muestran con una incomodidad: las reescrituras humanas “may lack sufficient information for optimal retrieval”, y las del modelo, más informativas, les ganan, sobre todo con un buscador léxico como BM25. En nuestra conversación empatan, pero las del modelo traen el número de reglamento, que la persona no escribió.

El protocolo. Cuatro formas de buscar sobre los turnos 2 a 6 de la conversación, que tienen los cuatro casos —elipsis, referencia, continuidad y cambio de tema—: el último turno solo, la conversación entera, la reescritura del modelo y una escrita a mano. Se mide si el acto correcto entró entre los k pasajes traídos (hit@k), por tipo de dependencia y por número de turno: la misma medida determinista de RAG, aplicada a la conversación. En clase se ve sobre una conversación; la versión estadística, con muchas, es parte del segundo proyecto.



el último turno acierta la elipsis y el cambio de tema; la conversación entera falla justo el cambio de tema

se mide la recuperación, no la redacción

La mejor reescritura no es la que mejor se lee: es la que trae el pasaje correcto. A veces la del humano pierde. ▸ Notebook §2 — las cuatro formas, sobre una conversación.

Estamos acá: la siguiente pregunta

Lo que resolvimos. El modelo ya ve la conversación: el harness le re-envía la lista entera en cada llamada. Y el buscador ya no busca con la frase suelta: busca con la pregunta reescrita, que se sostiene sola y trae el reglamento correcto.

Lo que costó. Una llamada más al modelo en cada turno, la del reescritor, y una falla nueva que no avisa: una reescritura prolija que busca otra cosa.

El problema nuevo. El reescritor y el generador leen el historial, y el historial viaja entero en cada turno: en el turno 20, el último de la conversación, cada llamada carga los veinte anteriores. Es lo que hace Claude con cada mensaje, y por eso un chat largo gasta más.



Primera respuesta a la pregunta madre: vuelve a entrar todo lo que se dijo. Funciona, y en cada turno se paga entero otra vez.

El turno 20 manda veinte turnos

Ya sabemos con qué buscar en cada turno. Ahora, lo que se manda: la conversación entera, otra vez, cada vez. La ayuda de Claude lo avisa a su manera: las conversaciones largas consumen más del límite de uso, y conviene empezar un chat nuevo para un tema nuevo.

Cada mensaje se paga muchas veces. Si cada turno suma 300 tokens, el turno 1 manda 300; el 2 manda 600 —el 1 otra vez, más el nuevo—; el 20 manda 6.000. Al final la conversación mide 6.000, pero se procesaron 300 + 600 + … + 6.000 = 63.000: el primer mensaje se pagó veinte veces, y el último, una. Cada envío es un prefijo de la conversación —todo lo dicho hasta ahí—, y se pagan todos.

Y crece con el cuadrado. Con el doble de turnos, cada envío es el doble de largo y hay el doble de envíos: cerca de cuatro veces más. En nuestra conversación, el turno 20 manda 6.148 tokens, y para llegar ahí se procesaron 80.334: 13 veces más. En un servicio pago es plata; en una GPU propia, tiempo y memoria.



cada fila es un prefijo de la conversación; el área de la escalera es lo que se paga

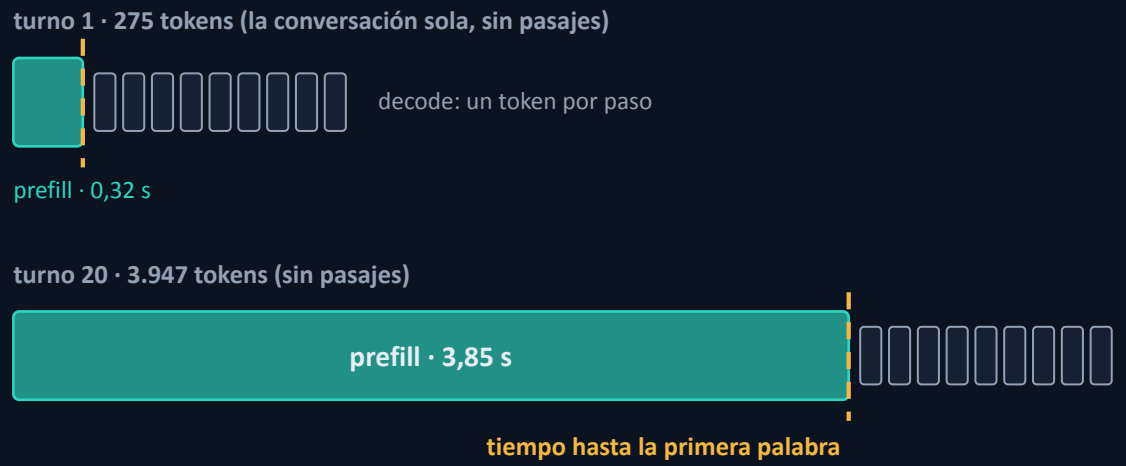
En el turno 20 la conversación mide 6.148 tokens, pero ya costó 80.334. Cada turno vuelve a pagar todo lo anterior, y el total crece con el cuadrado de los turnos. ▸ Notebook §3 — el tamaño del prompt por turno, en una conversación guionada de 20 turnos.

Leer y escribir son dos trabajos distintos

Generar una respuesta tiene dos fases. La primera lee el prompt: todos sus tokens a la vez, en paralelo, que es lo que una GPU hace bien. Se llama prefill (la fase del prompt). La segunda escribe la respuesta: un token por vez, y cada token es una pasada completa por la red, que depende del anterior. Se llama decode, y no se puede paralelizar.

Las dos fases se traban en cosas distintas. Un paper sobre cómo servir modelos, el de PagedAttention, lo describe así: la fase del prompt aprovecha el paralelismo de la GPU; la de generación la subutiliza y queda limitada por la memoria, no por el cómputo: para escribir cada token hay que leer todos los pesos de la memoria de la placa, y eso tarda más que las cuentas.

Dónde se paga el historial. Re-enviar la conversación entera se paga en el prefill, y el prefill pasa antes de la primera palabra. Cuanto más largo el historial, más espera el usuario para ver que algo empieza a salir: el tiempo hasta el primer token crece con la conversación. En el notebook, midiendo sólo la conversación —sin los pasajes—, el prefill pasa de 0,32 s a 3,85 s; escribir cada token cuesta siempre unos 33 ms.



medido en una T4: el prefill crece con el historial; el decode, no

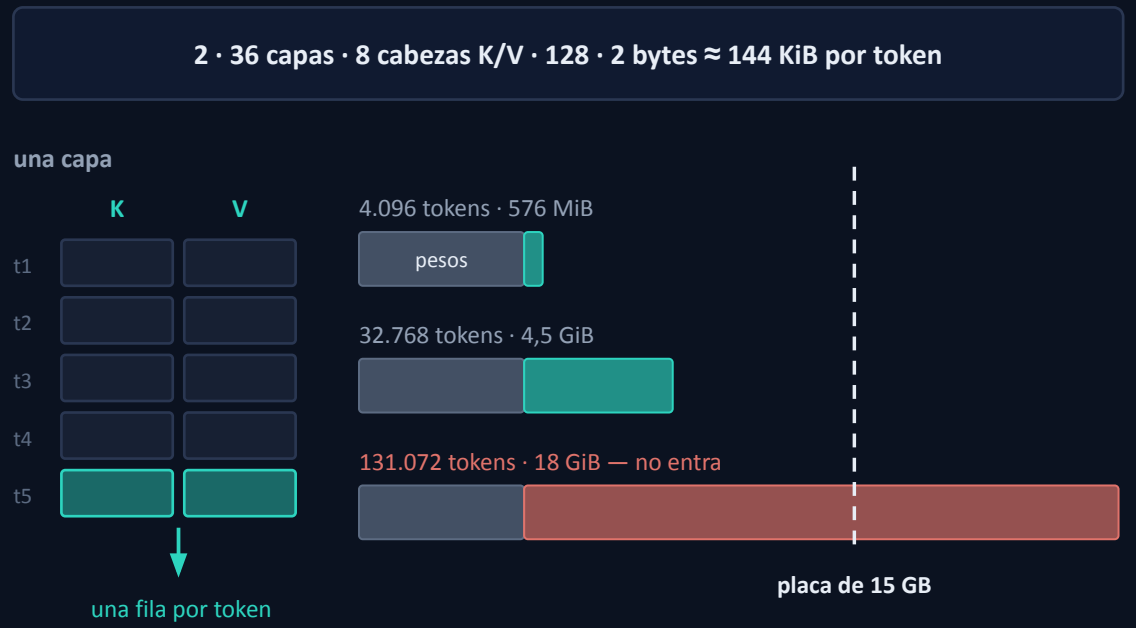
El historial se paga antes de la primera palabra. En cada turno, el modelo vuelve a leer la conversación entera. ▸ Notebook §3 — el tiempo de prefill y el de generación, a medida que crece el historial.

El KV cache: lo que guarda mientras escribe

Para escribir cada token, la atención necesita todo lo anterior. Como vimos en Transformers, cada token nuevo compara su consulta con las claves de todos los previos y promedia sus valores, en cada capa. Recalcular esas claves y valores en cada paso sería tirar trabajo: los de los tokens ya leídos no cambian. Así que se guardan: es el cache de claves y valores (key-value cache, o KV cache).

Tiene un tamaño que se puede calcular. Por token: dos vectores (clave y valor) × capas × cabezas de clave y valor × dimensión de la cabeza × bytes por número. En el modelo que usamos da 144 KiB por token (un KiB son 1.024 bytes): una ventana grande ocupa varios gigas, en la misma placa que los pesos —unos 5 GB, porque Ollama los usa cuantizados a unos 4 bits—.

La ventana es, físicamente, este cache. Quien sirve modelos lo administra como un sistema operativo la RAM: en bloques de tamaño fijo, como páginas. Es la idea de PagedAttention, de donde sale vLLM, uno de los servidores de modelos más usados: la analogía de la computadora, al pie de la letra.



medido: 5,20 GiB con la ventana de 4.096 y 9,16 GiB con la de 32.768

La memoria de trabajo tiene tamaño físico: cada token de la ventana ocupa 144 KiB de GPU. Por eso la ventana no es gratis aunque entre. ▸ Notebook §3 — cuánta memoria toma el modelo con la ventana en 4K y en 32K.

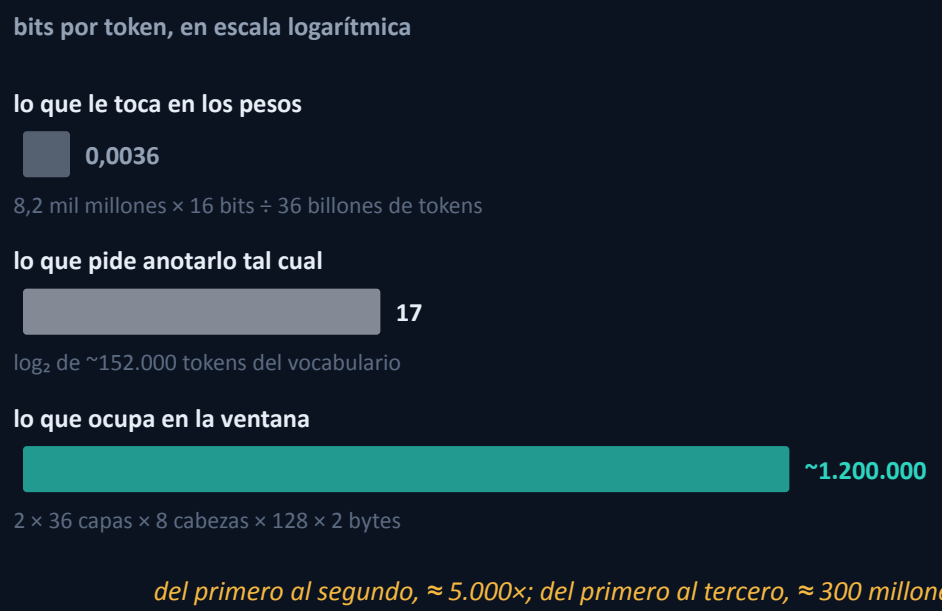
PARA PROFUNDIZAR · PESOS Y VENTANA EN BITS

Las dos memorias, en bits

Los pesos no tienen lugar para el texto. Volviendo a las dos memorias del principio: el modelo que usamos tiene 8.200 millones de parámetros, de 16 bits en el original: unos 131.000 millones de bits. Se entrenó leyendo unos 36 billones de tokens. Si se repartieran los bits de los pesos entre los tokens leídos —suponiendo, de más, que cada bit guardara texto—, a cada token le tocarían 0,0036 bits. Anotar un token tal cual, sin comprimir, pide unos 17 bits: casi cinco mil veces más. Lo que queda adentro son regularidades.

La ventana le da a cada token mucho lugar. Mientras un token está en la ventana, el KV cache guarda su clave y su valor en cada capa: 144 KiB por token, más de un millón de bits, en memoria de la GPU.

El contraste. Un token en la ventana ocupa unas 300 millones de veces lo que le toca a un token de entrenamiento en los pesos. No mide cuánta información tiene un token —la ventana guarda una representación rica y redundante, no 17 bits—: mide fidelidad y costo. Lo de la ventana está entero y es caro; lo de los pesos, comprimido y ya pagado.



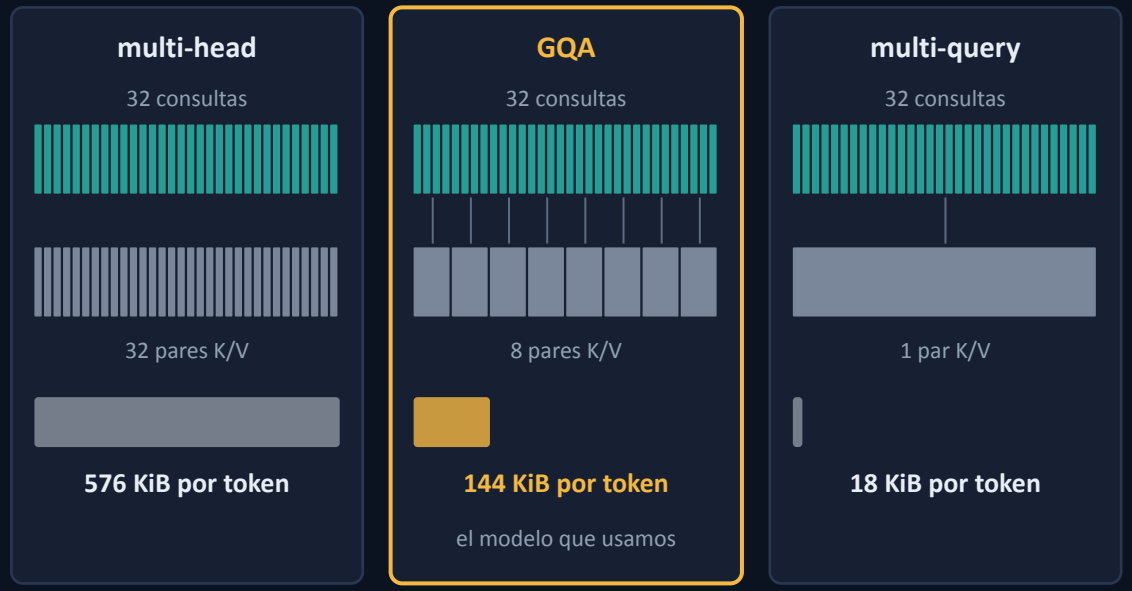
Los pesos no tienen lugar para guardar el texto; la ventana le da a cada token cientos de millones de veces más. Por eso lo que está en la ventana le gana a lo que está en los pesos. ▸ Notebook §3 — los dos números, calculados desde la configuración del modelo.

Por qué ocho cabezas y no treinta y dos

En la fórmula aparece un 8, y no es casual. En la atención multi-head que armamos en Transformers, cada cabeza tiene sus propias consultas, claves y valores. El modelo que usamos tiene 32 cabezas de consulta; si cada una tuviera su par de clave y valor, el cache sería cuatro veces mayor: 576 KiB por token en vez de 144.

La solución: compartir. En la atención de consultas agrupadas (grouped-query attention, GQA), las cabezas de consulta se juntan en grupos que comparten una misma clave y un mismo valor; acá, grupos de cuatro. El extremo es que todas compartan una sola (multi-query attention). GQA queda en el medio, con “quality close to multi-head attention with comparable speed to MQA”, según su paper.

Para comparar. El modelo de 13.000 millones de parámetros que usa de ejemplo el paper de PagedAttention, sin GQA, necesita 800 KB por token. Achicar el cache es una decisión de diseño de la arquitectura.



las consultas no se tocan: lo que se comparte es la clave y el valor

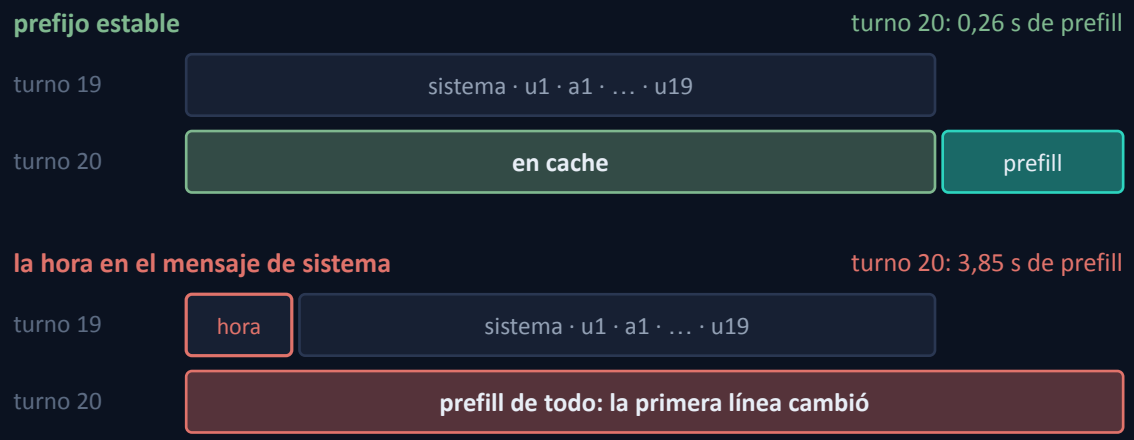
El tamaño de la memoria de trabajo es una decisión de arquitectura. Compartir claves y valores entre cabezas lo achica cuatro veces, casi sin pérdida.

No volver a leer lo que ya se leyó

Entre un turno y el siguiente, casi todo el prompt se repite. El turno 20 es el 19 más dos mensajes. El KV cache se arma en cada llamada y, cuando la respuesta termina, se tira. Si el servidor lo guarda, reusa lo que coincide y hace prefill sólo de lo nuevo: es el cache de prefijos (prefix caching; los proveedores lo venden como prompt caching). En la escalera del costo, de cada fila se lee sólo el escalón nuevo, y lo calculado en el turno 1 sigue sirviendo en el 20. Y no es memoria: el servidor guarda una cuenta, no un recuerdo; si el cache se pierde, la respuesta es la misma, sólo tarda más.

La condición es dura: idéntico, desde el principio. El cache sirve hasta el primer token que cambia; de ahí en más, se recalcula. Un timestamp en el mensaje de sistema, un turno viejo editado o un resumen reescrito arriba invalidan todo lo de abajo. Un equipo de agentes llama a la tasa de aciertos del cache "the single most important metric": en un proveedor, el millón de tokens en cache cuesta 0,30 dólares, contra 3. Es la táctica que vimos en prompt engineering —maximizar el cache hit rate—, ahora con su mecanismo.

Y un aviso para lo que viene. Recortar o resumir el historial reescribe el prefijo y rompe el cache: cada recorte o resumen cuesta releer todo. Achicar el historial es un compromiso, no un default. Y el cache tiene techo: la ventana.



los 20 turnos: 4,9 s de prefill con prefijo estable, 40,6 s con la hora

Nota de laboratorio: Ollama reusa el prefijo mientras el modelo siga cargado; su conteo incluye los tokens del cache, así que el ahorro se ve en el tiempo, no en el conteo.

el cache sirve hasta el primer token que cambia

Con un prefijo estable, re-enviar la conversación casi no cuesta. Cambiar una coma arriba invalida todo lo de abajo. ▸ Notebook §3 — prefijo estable contra timestamp en el mensaje de sistema, prefill por turno.

LA VENTANA SE ACABA · LA FALLA

Cuando no entra, algo se cae

El turno 20 ya no entra, y nadie avisa. El cache tiene techo: la ventana. El servidor asigna 4.096 tokens, y el prompt del turno 20 mide más de 6.000. A "¿En qué ciudad te dije que trabajo?", el modelo contesta, con toda fluidez: "No se menciona ninguna ciudad específica". Para que entrara, el servidor tiró sin avisar los mensajes más viejos, y con ellos el turno 0, donde estaba Vigo. Con una ventana el doble de grande, contesta Vigo.

Desbordar no le pasa al modelo: es una política de quien lo sirve. Desbordar es mandar más de lo que entra; quien sirve el modelo elige qué hacer:

Rechazar: un error, y no corre nada. La API de Anthropic contesta "prompt is too long"; Claude, en el chat, avisa. Ruidoso, pero no pierde nada en silencio.

Tirar lo más viejo (first in, first out): se pierden los primeros turnos, donde el usuario dice quién es y qué necesita.

Cortar el prompt —el principio, el medio o el final—: se pierden instrucciones o pasajes.

El modelo contesta con fluidez igual. Parece que «se olvidó», y en realidad el dato nunca le llegó. Como en Memento: no sabe qué olvidó.



medido, ventana de 4.096: 6.170 tokens llegaron como 4.081, y "Vigo", dicho en el turno 0, no llegó

Nota de laboratorio: Ollama usa las silenciosas: un chat pierde los más viejos (no el de sistema); un prompt único, el medio; si se llena mientras escribe, tira el principio y sigue.

Desbordar no es un error del modelo: es una política de quien lo sirve. La peligrosa es la silenciosa: parece que el modelo se olvidó, y en realidad el dato nunca le llegó. ▸ Notebook §4 — el desborde, provocado a propósito.

Dos límites: el del modelo y el de quien lo sirve

¿Por qué no entraba, si el modelo acepta 32.768 tokens? Porque el techo no es uno solo. El modelo tiene un máximo que aprendió en el entrenamiento: posiciones hasta cierto largo. Pero quien sirve el modelo decide cuánta ventana reservar por pedido, porque cada token de ventana ocupa memoria de la placa: los 144 KiB del KV cache. Un modelo que acepta 32.768 tokens puede estar sirviéndose con muchos menos: en nuestra placa, 4.096.

El límite efectivo es el más chico de la cadena. Es la regla que ya apareció en RAG, cuando el embedder cortaba sin avisar los pasajes que no le entraban: el límite de un pipeline es el de su componente más angosto. Acá la cadena es el máximo del modelo, la ventana que asigna el servidor y, adentro, lo que queda para el prompt.

Y la respuesta también ocupa ventana. La documentación de Anthropic define la ventana como todo lo que el modelo puede referenciar al generar, “including the response itself”. Un prompt que llena la ventana no deja lugar para contestar: con una ventana de 4.096 y un prompt de 4.000, quedan 96 tokens para la respuesta.



el límite efectivo es el más chico

Nota de laboratorio: Ollama asigna por defecto 4.096 tokens en GPUs de menos de 24 GiB, como una T4, y su endpoint compatible con OpenAI no deja cambiarla por pedido; la documentación vieja dice 2.048: se mide, no se cree. La configuración del modelo, que imprime el notebook, dice 40.960 posiciones: 32.768 reservadas para respuestas largas de razonamiento y 8.192 para el prompt.

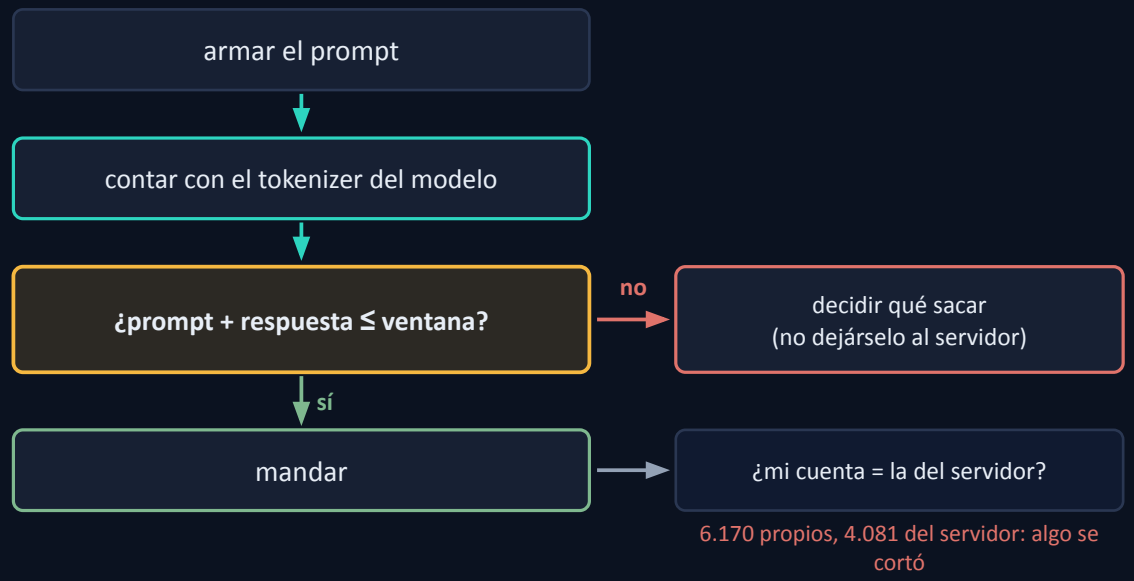
La ventana que importa no es la del modelo: es la reserva de memoria que hace quien lo sirve. Y la respuesta también tiene que entrar.

Contar antes de mandar

La defensa es aburrida y funciona. Antes de mandar, contar los tokens del prompt con el tokenizer del propio modelo —se mide en tokens, no en caracteres—, sumar el lugar reservado para la respuesta y comparar con la ventana. El largo de la respuesta no se conoce de antemano: se suma un tope que fija el harness y que el servidor hace cumplir. En el turno 20: 6.170 + 600 = 6.770, contra 4.096. No entra, y el harness lo sabe antes de mandar.

Mejor una falla ruidosa que una silenciosa. Si al servidor se le puede pedir que rechace en vez de recortar, se le pide: el nuestro, así, contesta con un error en vez de tirar a Vigo. Y después de mandar, se compara la cuenta propia con la del servidor: 6.170 contra 4.081, se cortaron más de 2.000 tokens sin aviso. Las APIs comerciales traen una función para contar tokens justamente para esto.

La semilla de lo que sigue. Si el harness cuenta, el harness decide qué se cae, y deja de decidirlo el servidor. Es la idea de repartir la ventana, después del recreo.



Nota de laboratorio: en Ollama, truncate: false y shift: false convierten los recortes silenciosos en errores explícitos.

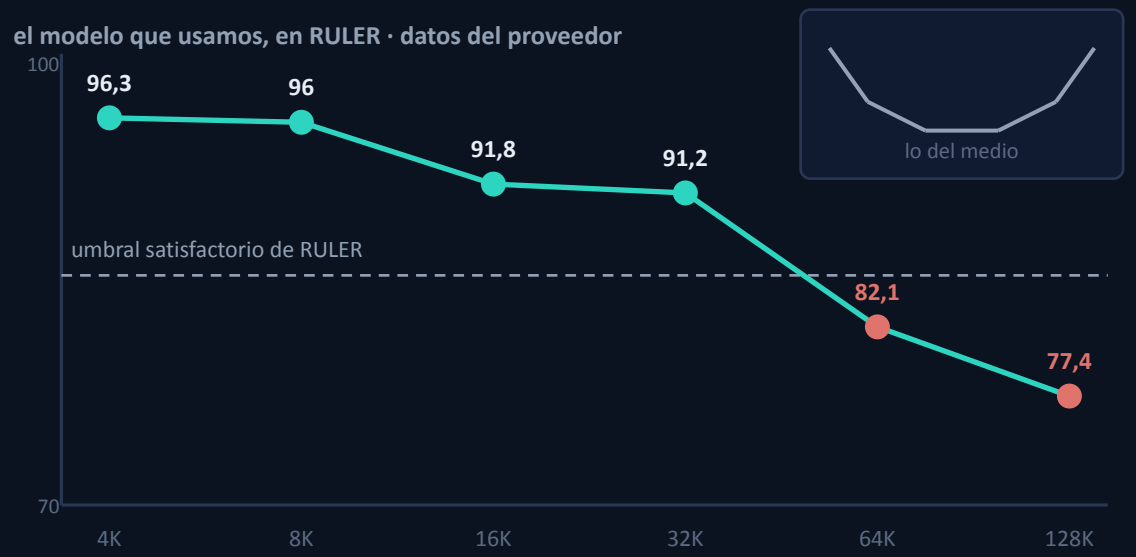
Si el harness no cuenta, recorta el servidor, y recorta lo que no debía. Contar convierte una falla silenciosa en una decisión. ▸ Notebook §4 — la cuenta propia contra la del servidor.

Que entre no quiere decir que se use

Hasta acá, el problema era que no entrara. Hay uno más sutil: que entre y no se use bien. Cuando vimos prompt engineering aparecieron la curva en U de Lost in the Middle y el Context Rot: el modelo aprovecha peor lo del medio, y su confiabilidad cae con el largo. Los benchmarks de contexto largo lo midieron a escala.

La ventana declarada no es la efectiva. El fabricante del modelo que usamos publica su propia caída en RULER, un benchmark que esconde datos en textos largos y pregunta por ellos: de 96 sobre 100 con 4.000 tokens a 77 con 128.000, y más baja en modo de razonamiento. Medido en muchos modelos: en RULER, sólo la mitad de los que declaran 32.000 tokens rinde bien a ese largo; en NoLiMa, cuando la pregunta no comparte palabras con el dato, casi todos se caen. La documentación de Anthropic le pone nombre: context rot.

Por qué importa más para la memoria. En una conversación, el usuario rara vez pregunta con las mismas palabras con que dijo el dato: dijo “trabajo en una pesquera de Vigo”, y más adelante pregunta “¿qué normas rigen en mi puerto?”. Es el caso que mide NoLiMa, y el que más cae.



a 64K y 128K cae por debajo del umbral: entra, pero no se usa bien

RULER: la mitad de 17 modelos rinde bien a 32K · NoLiMa: 11 de 13 caen a menos de la mitad

La ventana tiene dos tamaños: el que acepta y el que usa bien. Para la memoria importa el segundo, y es más chico.

Por qué hay un techo

Tres razones, y las tres ya las vimos.

Cómputo: la atención compara cada token con todos los anteriores, así que esa parte del prefill crece con el cuadrado del largo. Es otro cuadrado, distinto del de la escalera: éste pasa adentro de una sola lectura.

Memoria: el KV cache crece token a token, 144 KiB cada uno, en la misma placa que los pesos.

Calidad: en el transformer de la clase, la posición era una tabla de block_size filas, y más allá no había nada. Los modelos actuales la codifican como una rotación (RoPE), sin tabla: pueden seguir, pero más allá del largo con que se entrenaron entienden peor.

Una ventana más grande corre el techo, no lo saca. Hay modelos con ventanas de un millón de tokens, y siguen pagando las tres cosas: cuestan más, ocupan más y rinden peor a medida que se llenan.

La conclusión que ordena el resto de la clase. La ventana no es un contenedor donde se tira todo lo que entre: es un presupuesto. Y si es un presupuesto, alguien lo tiene que repartir.



más ventana = más costo, más memoria, menos calidad

Una ventana más grande corre el techo, no lo saca. La ventana es un presupuesto.

Quién gasta la ventana

Si la ventana es un presupuesto, primero hay que saber quién lo gasta. En un turno de RAG conversacional, la ventana tiene seis inquilinos, y cada uno viene de un lugar distinto.

Instrucciones: el mensaje de sistema; son fijas.

Pasajes: lo que trae el buscador; dependen de cuántos y de cuán largos.

Historial: la conversación hasta acá; crece en cada turno.

Memoria: lo que se sepa de otras sesiones; vacía hasta la semana que viene.

Pregunta: el turno de ahora.

Reserva para la respuesta: no es contenido, pero ocupa lugar.

Crecen por razones distintas. Todos se deciden, menos uno: el historial se acumula, hasta que choca con el presupuesto. En el turno 20, el historial completo mide unos 3.750 tokens y entran 1.707: once turnos quedan afuera.



el historial toca su tope y empuja turnos afuera · memoria: 0

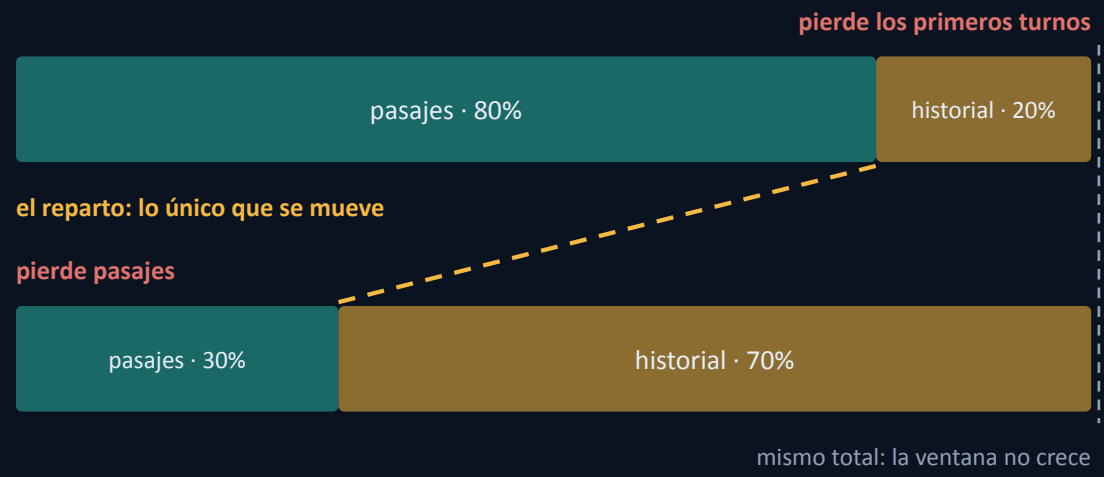
En una conversación con RAG la ventana tiene seis inquilinos, y el que crece sin parar es el historial. ▸ Notebook §5 — los tokens de cada inquilino, turno a turno.

El reparto: pasajes contra historial

En RAG, cuántos pasajes traer era un presupuesto, no un número libre. Ahora el presupuesto es compartido: con la ventana fija, cada token de historial es un token menos para pasajes, y al revés.

Qué compra cada lado. Más historial: el modelo resuelve las referencias (“ese reglamento”, “el bacalao”) y no pierde las restricciones que el usuario dijo al principio. Más pasajes: más chance de que entre el correcto. Pero no sin techo: en Lost in the Middle, traer más de 20 documentos sumó apenas un 1,5% de acierto, porque “reader model performance saturates long before retriever performance saturates”: el buscador seguía encontrando más, y el modelo ya no lo aprovechaba.

El reparto no tiene una respuesta general. Depende del corpus, de cuánto dependen los turnos unos de otros, de cuán largos son los pasajes. Se fija un total, se mueve una sola cosa —el reparto— y se mide. En el notebook, los pasajes tienen un tope de 1.600 tokens —los cinco de siempre— y el historial se queda con lo que sobra. Medir otros repartos es parte del segundo proyecto.



se fija el total, se mueve el reparto, se mide

Con la ventana fija, cada turno de historial le saca lugar a un pasaje. El reparto no se adivina: se mide.

Armar la ventana en cada turno

El harness deja de pasar una lista que crece. En vez de mandarle al servidor el historial entero y esperar que entre, el harness arma la ventana en cada turno, adentro de un presupuesto contado. A esa pieza la llamamos el ensamblador, y sigue un orden:

Primero, lo fijo: instrucciones, pregunta y reserva para la respuesta.

Después, los pasajes, del mejor al peor, hasta su tope.

Después, el historial, del turno más nuevo al más viejo, hasta lo que quede.

Por último, contar y mandar. Es lo que el harness de Claude hace en cada mensaje sin que se vea; con la API, lo escribimos nosotros.

Tirar turnos viejos pasa a ser una decisión. Lo que antes recortaba el servidor sin avisar ahora lo decide el harness, a la vista y con un criterio. El orden también se diseña: lo estable arriba, para que el cache de prefijos aguante; lo que cambia en cada turno, abajo; lo importante, en los extremos, por la curva en U.

Es el dispositivo central de la clase. Desde acá, cada técnica —recortar, resumir, comprimir, traer memorias— es una regla nueva del ensamblador: qué entra a cada segmento y en qué forma.



la ventana armada: arriba lo estable, abajo lo que cambia en cada turno

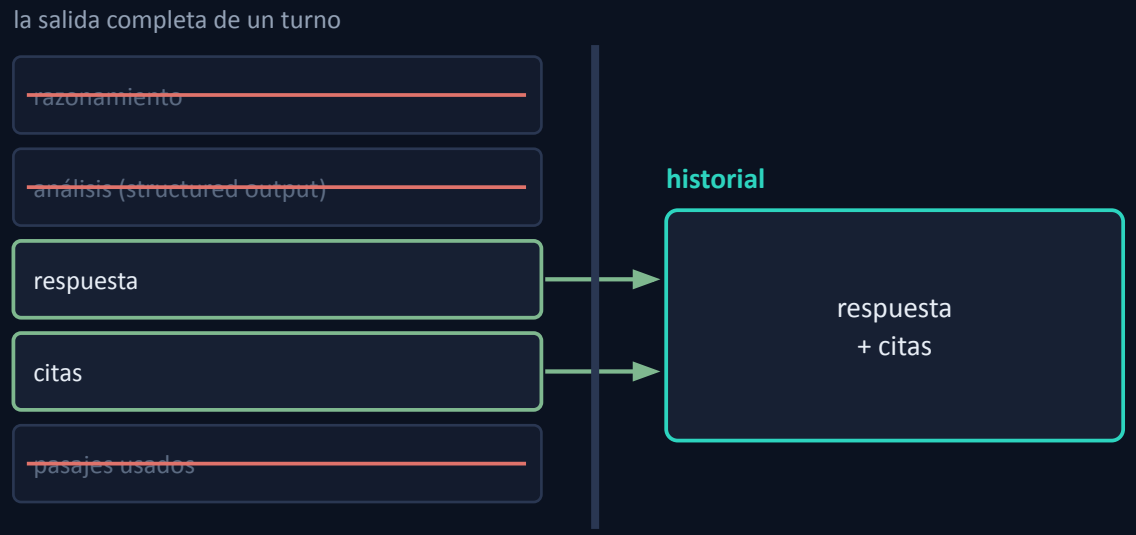
La ventana no se deja crecer: se arma en cada turno, adentro de un presupuesto contado. Desde acá, cada técnica es una regla nueva del ensamblador. ▸ Notebook §5 — un ensamblador chico que imprime cada segmento.

El historial guarda lo que se dijo

Qué entra al historial también se decide. La salida de un turno trae más que la respuesta: puede traer el razonamiento del modelo, un campo de análisis de un structured output, los pasajes que usó, las citas. Al historial va lo que vio el usuario: la respuesta y sus citas.

Por qué no el razonamiento. Un razonamiento guardado se vuelve a pagar en cada turno, y lo peor: después se lee como si se hubiera dicho. Una conjetura del modelo (“el usuario probablemente se refiere a…”) pasa a ser, para el turno siguiente, un hecho de la conversación. Los proveedores también lo deciden: la API de Anthropic saca el razonamiento de los turnos anteriores en algunos de sus modelos.

Por qué no los pasajes. Se recuperan otra vez en cada turno, con la pregunta de ese turno. Guardarlos en el historial los duplica, le come el presupuesto a otro inquilino, y quedan ahí aunque la pregunta ya sea otra.



lo tachado no entra: ni se vuelve a pagar, ni se lee como dicho

Nota de laboratorio: el fabricante de nuestro modelo recomienda no guardar el razonamiento en el historial, y su plantilla de chat ya lo saca.

El historial es lo que se dijo, no lo que se pensó. Un razonamiento guardado se cobra en cada turno, y mañana se lee como si alguien lo hubiera dicho.

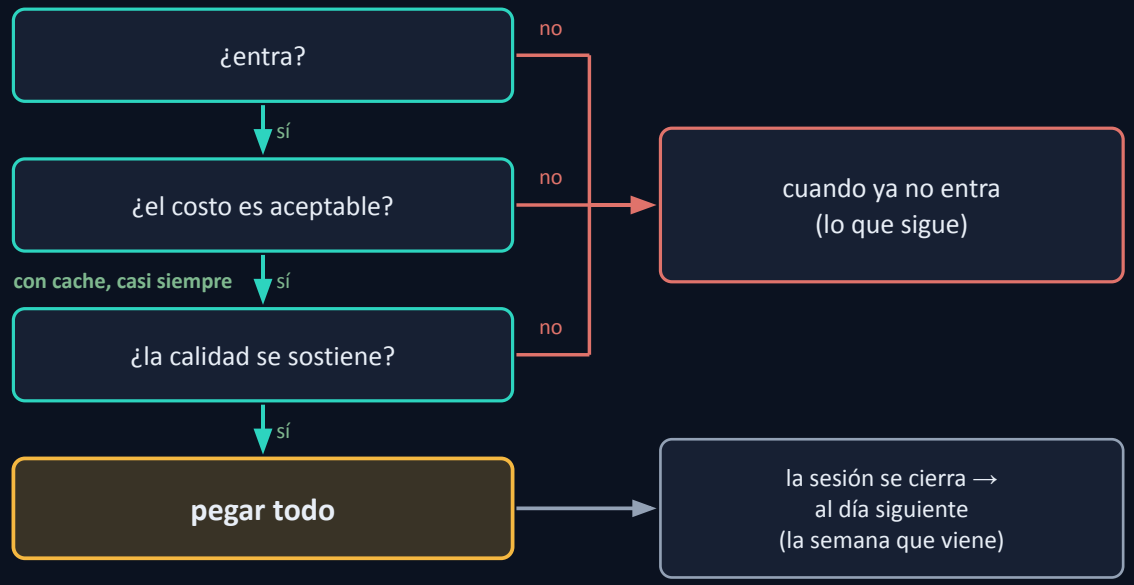
Si entra, se pega

El criterio antes de cualquier técnica de memoria. Si el historial entero entra en el presupuesto, y el costo y la calidad aguantan, re-enviarlo es la vara a superar. Es el mismo criterio con el que arrancaba RAG: si el documento entra, se pega.

La evidencia viene de un proveedor de memoria, contra su propio producto. Mem0 es un sistema de memoria de largo plazo, de los que vienen la semana que viene. En su propio paper, sobre conversaciones largas que todavía entraban en la ventana, mandar la conversación completa acertó más que su memoria: 72,9% contra 66,9% de respuestas correctas. Los autores lo dicen: el contexto completo da "a slight accuracy edge". Lo que la memoria compró fue costo y velocidad: unos 1.800 tokens en vez de 26.000, y casi todas las respuestas en menos de 1,4 s, contra 17.

Y pegar sale más barato de lo que parece. Con cache de prefijos, lo que se re-envía sale casi todo del cache: a un décimo del precio, y mucho más rápido —en el notebook, el turno 20 tarda 0,26 s en vez de 3,85—. La ventaja de costo de una memoria se achica.

Cuándo dejar de pegar. Cuando no entra, cuando cuesta demasiado aun con cache, cuando la calidad cae con el largo, o cuando tiene que sobrevivir a la sesión.



cada "no" es una razón distinta para dejar de pegar

Antes de cualquier técnica de memoria, la vara es re-enviar todo. Una memoria tiene que ganarle a eso, y donde el historial entra, a veces no le gana.

Estamos acá: la ventana ya no alcanza

Lo que resolvimos. Sabemos cuánto cuesta re-enviar todo y cómo abaratarlo: un principio del prompt estable, que el servidor no vuelve a leer. Sabemos dónde se acaba la ventana y que, si nadie cuenta, el servidor recorta en silencio. Y el harness ahora arma la ventana en cada turno, adentro de un presupuesto contado.

Lo que costó. Contar en cada turno y decidir un reparto entre pasajes e historial: la ventana dejó de ser un balde y pasó a ser un presupuesto.

El problema nuevo. En el turno 20, con una ventana de 4.096 tokens, el ensamblador ya deja once turnos afuera, y entre ellos están el 0 y el 7: Vigo y el artículo 9. Ahora lo decide el harness, a la vista, y no el servidor en silencio. Pero tirar los más viejos es sólo una forma de sacar algo, y no la mejor.



en el turno 20, con la ventana de 4.096 tokens: 11 turnos afuera, del 0 al 10

0 · Vigo y 7 · art. 9: los dos datos plantados, afuera

lo decide el harness, a la vista: falta decidir qué se saca

Segunda respuesta a la pregunta madre: quién decide. Ya no lo decide el servidor en silencio: lo decide el harness, a la vista. Falta decidir qué se saca.

Olvidar es una decisión

El ensamblador ya está olvidando. Tira los turnos más viejos: en el turno 20 dejó afuera el 0, donde el usuario dijo Vigo, y el 7, donde anotó el artículo 9. Eso ya es una política de olvido —la regla que decide qué sale de la ventana—, aunque nadie la haya elegido.

Toda política de olvido es una apuesta sobre qué va a importar después. Hay cuatro formas de hacer lugar: tres sacan turnos del historial y una achica los pasajes.

Recortar: tirar los turnos viejos; apuesta a lo reciente. Su variante, anclar, deja fijo el primero.

Resumir: condensarlos; apuesta a lo que el resumidor —el mismo modelo, en otra llamada— cree importante.

Comprimir los pasajes: achicar el otro inquilino; apuesta a lo que un puntaje marca como informativo.

Descargar: guardar los turnos enteros afuera, con un puntero. No destruye nada: decide al leer, con la pregunta delante, y su riesgo es no encontrarlo.

En el mapa se enciende la segunda flecha, comprimir: achicar lo que hay en la ventana, que es lo que hacen recortar, resumir y comprimir los pasajes. Descargar es otra flecha, escribir.



Toda política de olvido es una apuesta sobre qué va a importar, hecha antes de saber la pregunta.

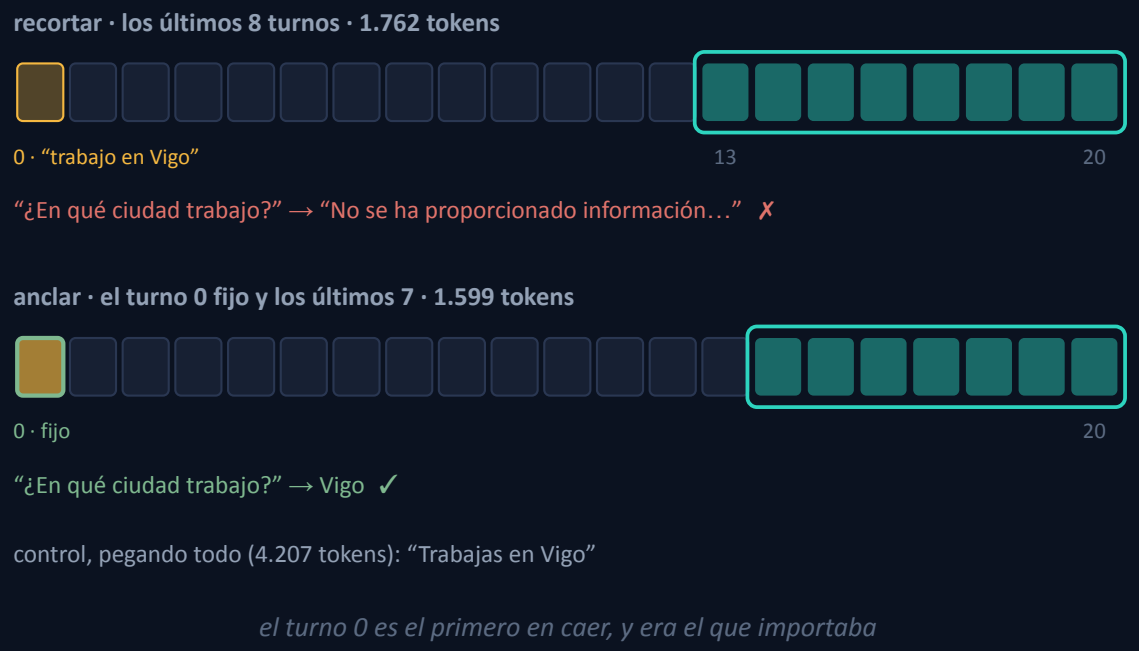
Recortar: quedarse con lo último

La política más simple: una ventana deslizante. Se conservan los últimos N turnos —o T tokens— y lo anterior se tira. Es lo que ya hace nuestro ensamblador: gratis, predecible, sin llamadas extra, y casi siempre acierta, porque en una conversación se usa lo que se acaba de decir.

La prueba. Después del turno 20, el usuario pregunta "¿En qué ciudad trabajo?", que se contesta con el turno 0. No busca pasajes, así que compara recortar, anclar, resumir y descargar. El historial tiene lugar para ocho turnos, más o menos lo del reparto: quedan afuera del 0 al 12.

Y olvida justo lo que se dijo primero, que muchas veces es lo más importante: "trabajo en una pesquera de Vigo". La guía de OpenAI lo nombra como el riesgo de recortar: "context loss". El parche es un ancla: el primer mensaje del usuario queda fijo. Pero ocupa lugar —sale un turno reciente— y sólo salva lo del principio: el artículo 9, del turno 7, se pierde igual. Para anclarlo había que saber que iba a importar.

Un detalle de cache. Tirar de a un turno cambia en cada turno el principio del prompt y rompe el cache de prefijos; tirar en bloques lo rompe menos.



Recortar es gratis y predecible, y olvida justo lo que se dijo primero. Fijar un ancla lo emparcha, no lo resuelve. ▸ Notebook §6 — un hecho plantado en el primer turno, preguntado después del turno 20.

Resumir: un resumen que se reescribe

En vez de tirar los turnos viejos, condensarlos. Cuando el historial pasa su presupuesto, una llamada al modelo resume los turnos que van a salir, y el resumen ocupa su lugar. La vez siguiente se actualiza: el modelo recibe el resumen anterior y los turnos que salen, y escribe el resumen nuevo. Es un resumen corrido (running summary), también llamado recursivo, y un trabajo sobre memoria de diálogos largos mostró que funciona (Wang et al.).

La versión que se usa: el summary buffer. Lo viejo, como resumen; lo reciente, textual: lo que se acaba de decir se conserva entero, y lo anterior queda condensado en vez de perdido.

Claude lo hace, y lo llama compactar. En los planes pagos, cuando un chat se acerca al límite de la ventana, Claude resume los mensajes viejos y la conversación sigue: eso es compactar (compaction). La propia Anthropic advierte que un resumen sigue siendo un resumen, y que a veces pierde detalles.

Lo que cuesta. Una llamada extra cada vez que se re-resume, y algo menos obvio: el resumen va arriba, cerca del principio del prompt, así que cada vez que cambia rompe el cache de prefijos. Y el resumen tiene su propio presupuesto —en la prueba, cien palabras—: no crece con la conversación, así que cada reescritura vuelve a decidir qué se cae. El riesgo que nombra la guía de OpenAI es otro, "context distortion": un resumen que dice algo un poco distinto de lo que pasó.



sin "Vigo", el modelo contestó que le faltaba información

Un resumen conserva lo que el resumidor creyó importante. Guarda más que recortar, y sigue siendo una apuesta: en la prueba, Vigo no sobrevivió. ▸ Notebook §6 — el mismo hecho plantado, ahora bajo un resumen.

Cuándo compactar

Cuándo se resume es un parámetro de diseño. Demasiado temprano, se tira detalle que todavía entraba; demasiado tarde, la calidad ya cayó por el largo —que entre no quiere decir que se use—, y la propia llamada que resume puede no entrar: tiene que leer todo lo que resume.

El diseño de MemGPT, con números. MemGPT es el sistema que tomó al pie de la letra la analogía de la computadora. Su ventana es una cola de mensajes encabezada por un resumen corrido. Cuando el prompt pasa el 70% de la ventana, el sistema le avisa al modelo que la memoria está bajo presión, para que guarde afuera lo que le importe; lo retomamos cuando el que decida sea el modelo. Al 100%, saca de una vez los mensajes más viejos hasta liberar la mitad de la ventana, y los pliega en el resumen. Sacar de a mucho tiene sentido: menos re-resúmenes y menos caches rotos.

Cada sistema elige su momento. Los frameworks y las APIs traen su propio umbral. Claude Code compacta automáticamente cuando se acerca al límite, y también cuando se le pide, con /compact. La misma gente de LangChain lo dijo así: “there are good times and bad times to compact”.



Cuándo compactar es un parámetro de diseño: de a mucho, pocas veces, y con aviso antes del límite.

PARA PROFUNDIZAR · CLEARING

Limpiar antes de resumir

No todo lo que hay en el historial vale lo mismo. Hay contenido que se puede volver a traer cuando haga falta: pasajes recuperados, resultados de una búsqueda, salidas de herramientas —lo que devuelve, por ejemplo, una búsqueda en la web o un archivo leído—. Y hay contenido que no se recupera de ningún lado: lo que se dijo en la conversación, empezando por lo que dijo el usuario.

Limpiar no es lo mismo que compactar. Anthropic las separa así:

Limpiar (clearing): sacar de la ventana los datos viejos que se pueden volver a buscar.

Compactar (compaction): resumir la ventana entera.

Claude Code las aplica en ese orden: primero limpia las salidas viejas de herramientas, y recién si no alcanza, resume la conversación.

En nuestro chat ya está hecho por diseño. Los pasajes de turnos anteriores nunca entran al historial: se recuperan de nuevo en cada turno. Lo que queda para comprimir es lo que se dijo, y ahí está lo que más cuesta perder.



lo gris se vuelve a buscar; lo ámbar no vuelve de ningún lado

Antes de resumir lo que se dijo, se tira lo que se puede volver a buscar. Lo que se dijo no vuelve de ningún lado.

PARA PROFUNDIZAR · PROMPT COMPRESSION

Comprimir los pasajes

El otro inquilino también se puede achicar. Los pasajes son los que RAG trajo para la pregunta de ahora: comprimirlos es un paso más de ese RAG, en la misma llamada, entre buscar y armar la ventana. Lo que dejan de ocupar se lo lleva el historial, sin tocar la conversación. Tres formas:

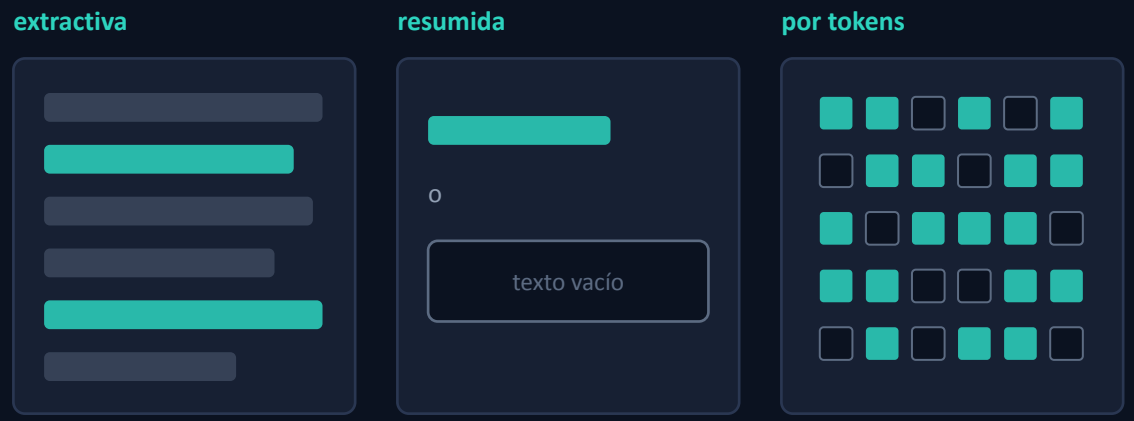
Extractiva: quedarse con las oraciones de cada pasaje que el re-puntuador —el mismo cross-encoder de RAG— puntúa más alto contra la pregunta.

Resumida: RECOMP resume los documentos recuperados hasta un 6% de su largo; cuando no ayudan, devuelve un texto vacío, y el modelo contesta con lo que sabe de sus pesos.

Por tokens: LLMLingua tira los tokens que un modelo chico adivina fácil: si se pueden adivinar, aportan poco. Comprime hasta 20 veces con poca pérdida, y el resultado es ilegible para una persona pero útil para el modelo.

La resumida es un resumen como el de la conversación, con una diferencia: el pasaje original sigue en el corpus, y el próximo turno lo vuelve a buscar entero. Más allá están los métodos que aprenden a comprimir el contexto en unos pocos vectores (gist tokens, ICAE); los nombramos y seguimos.

Lo que cuesta. Otra llamada a otro modelo, y un texto que ya no se puede citar tal cual. Nuestro usuario pidió las respuestas siempre con el artículo: comprimir el pasaje complica justo eso.



se achican los pasajes: la conversación queda intacta

Comprimir pasajes le hace lugar al historial sin tocar la conversación, y paga con otra llamada y con un texto que ya no se puede citar tal cual.

Descargar, no borrar

Recortar y resumir destruyen algo. Recortar tira turnos viejos; resumir los condensa con pérdida. Y lo que se dijo no vuelve de ningún lado: a diferencia de un pasaje comprimido, no hay un corpus de donde volver a buscarlo. Hay una forma de sacar turnos que no destruye nada: descargar (offloading). Lo que sale de la ventana se escribe entero en un archivo o en un almacén, y en la ventana queda un puntero, una línea que dice dónde quedó: “turnos 0 a 12 → historial_01.md”.

Lo que sale se puede volver a traer. Cuando una pregunta necesita un detalle de esos turnos, se busca en el archivo y el texto vuelve entero. En la prueba, “¿En qué ciudad trabajo?” trae de vuelta el turno 0, y Vigo llega a la ventana. Combinado con un resumen, compactar se vuelve reversible: en la ventana queda el resumen, y afuera, el original. Un equipo que construye agentes en producción lo tiene como principio —el sistema de archivos como contexto, “unlimited in size, persistent by nature”—, y los agentes de LangChain hacen lo mismo: resumen la ventana y guardan el historial completo en disco.

Y es el puente a la semana que viene. Traer de vuelta un pedazo de lo descargado es buscar en la propia conversación: RAG sobre lo que el sistema escribió. Que llegue tampoco quiere decir que se use bien: en la prueba, la respuesta arrancó copiando la vieja del turno 0, “Acuso recibo…”. En el mapa se enciende la tercera flecha, escribir, todavía punteada: lo descargado es de esta conversación, y un chat nuevo no lo ve.



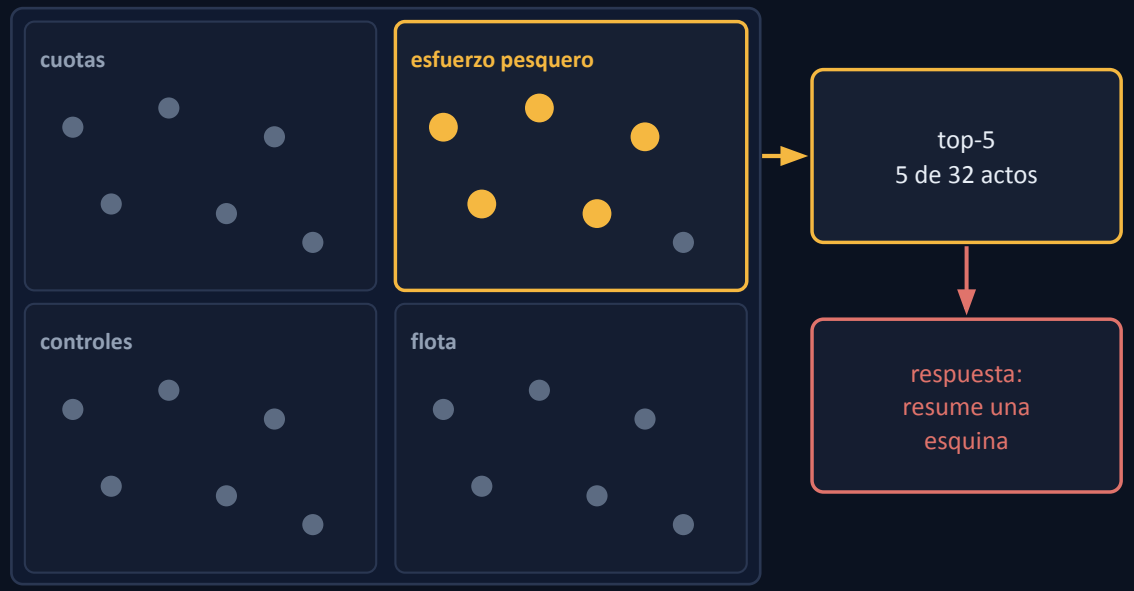
Lo que se descarga con un puntero se puede volver a traer. Es la primera memoria fuera de la ventana.

La pregunta que ningún pasaje responde

Hasta acá resumimos una conversación. El mismo problema —algo que no entra en la ventana— aparece con un documento largo o con un corpus entero. Es lo que pasa en un proyecto de Claude con muchos archivos: cuando no entran, Claude pasa a buscar en ellos con RAG. Este tramo se va un momento de la conversación al corpus, y después vuelve: lo que se aprende acá sobre resumir por partes es lo que le pasa al resumen corrido.

Las preguntas globales. Supongamos que el usuario de Vigo pide un panorama: “¿Cuáles son los temas principales de las regulaciones de pesca del corpus?”, o “¿de qué trata este reglamento?”. No hay un pasaje que las responda: la respuesta está repartida en todo el texto. Top-k trae k fragmentos de una esquina, y el generador resume esa esquina como si fuera el todo. En RAG, lo recuperado era el techo de la respuesta; acá ese techo es un rincón del corpus.

Tiene nombre en la literatura. El trabajo de Microsoft que presentó GraphRAG lo dice así: RAG “fails on global questions directed at an entire text corpus, such as 'What are the main themes in the dataset?'”.



la respuesta global está repartida en las cuatro zonas

Una pregunta global no tiene pasaje: su respuesta está repartida en todo el corpus, y top-k trae una esquina. ▸ Notebook §7 — una pregunta global sobre las regulaciones de pesca, con RAG y con una cadena de resumen.

Tres cadenas para lo que no entra

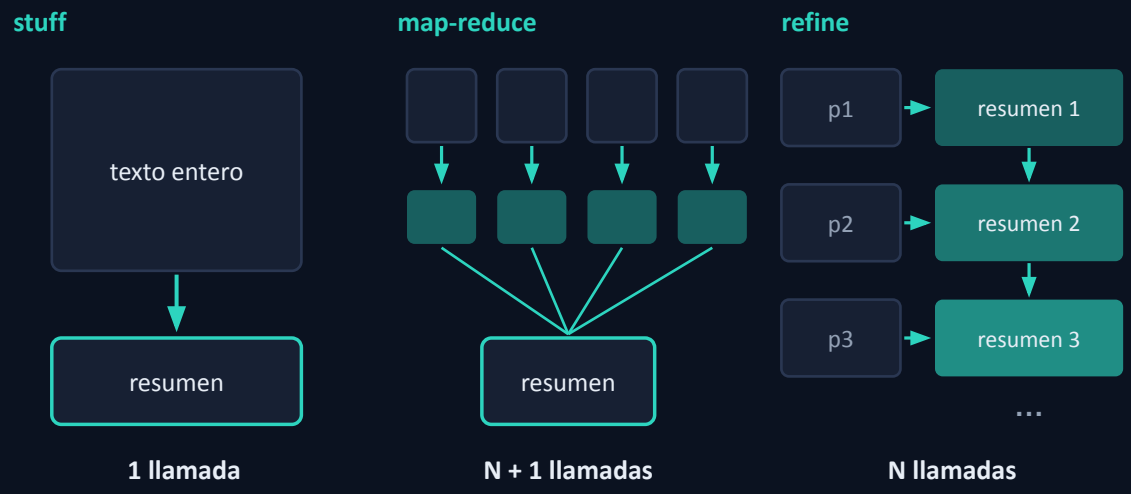
Si lo que hay que resumir no entra, se resume por partes. Pasa con el corpus, y pasa con la conversación: la propia llamada que resume los turnos viejos también tiene que entrar en la ventana. Para eso se encadenan llamadas al modelo —la salida de una es la entrada de la siguiente—, y hay tres cadenas canónicas:

Stuff (meter todo): el texto entero en una llamada. No parte nada: es el "si entra, se pega" de siempre, y la vara contra la que se miden las otras dos.

Map-reduce: se parte el texto, se resume cada pedazo por separado (el map) y después se resumen los resúmenes (el reduce).

Refine: se recorren los pedazos en orden, arrastrando un resumen que cada pedazo actualiza.

Lo que las separa. Map-reduce trabaja en paralelo y junta al final; refine trabaja en orden y arrastra. Cada una pierde algo distinto, y la diferencia se ve en las próximas dos láminas.



Nota de laboratorio: en LlamaIndex se llaman compact, tree_summarize y refine. Su modo simple_summarize "truncates all text chunks to fit": un recorte silencioso adentro de un framework.

Cuando el texto no entra, se resume por partes: en paralelo y después juntando, o en orden y arrastrando un resumen.

Map-reduce: en paralelo, y después juntar

Cada pedazo se resume sin saber de los otros, y por eso las llamadas pueden ir en paralelo.

Anda cuando la respuesta se arma sumando partes. "¿Cuáles son los temas?" se contesta pedazo por pedazo. Sobre fichas de los 32 actos de pesca —el título y el primer artículo de cada uno—, devolvió una veintena de temas, entre ellos los que RAG no vio: el Báltico, las aguas profundas, los descartes.

Y se paga en cada pregunta. Map-reduce corre cuando la pregunta llega: en la prueba, 5 llamadas y unos 70 segundos, y eso que leyó fichas. Alguien tiene que decidir que la pregunta lo merece —nuestro chat, en el turno 20, buscaría top-5 como siempre—. La otra salida es resumir una sola vez, antes de las preguntas.

Pierde lo que vive entre pedazos. Una relación que cruza el borde —un artículo que remite a otro, una excepción que se explica tres páginas antes— no está entera en ningún pedazo, así que ningún resumen parcial la ve.

La prueba, sobre un acto largo. 32015R0009, un reglamento de 2015 sobre subproductos animales: 6.752 tokens, 7 pedazos. Con la ventana en 8.192 entra entero, así que stuff sirve de control. Ninguna cadena conservó la relación que cruzaba pedazos —el artículo 1 suprime una excepción del reglamento de 2011, y el porqué está en los considerandos—, ni siquiera stuff: con un modelo chico, se pierde hasta lo que la cadena dejaba ver.



la relación cruza el borde: ningún resumen parcial la tiene

32015R0009 · subproductos animales	stuff	map-reduce	refine
qué modifica: el 142/2011	✓	✓	✓
la relación: suprime una excepción	✗	✗	✗
… y la vuelve permanente	✗	✗	✗
un detalle: hasta 20 kg por semana	✗	✗	✗
otro detalle: umbrales de sustratos	✗	✗	✗
se aplica desde el 23/2/2015	✓	✗	✗
transición hasta el 27/9/2015	✗	✗	✓

fechas: map-reduce dijo "22 de febrero"; refine, "6 de enero", la del título del acto

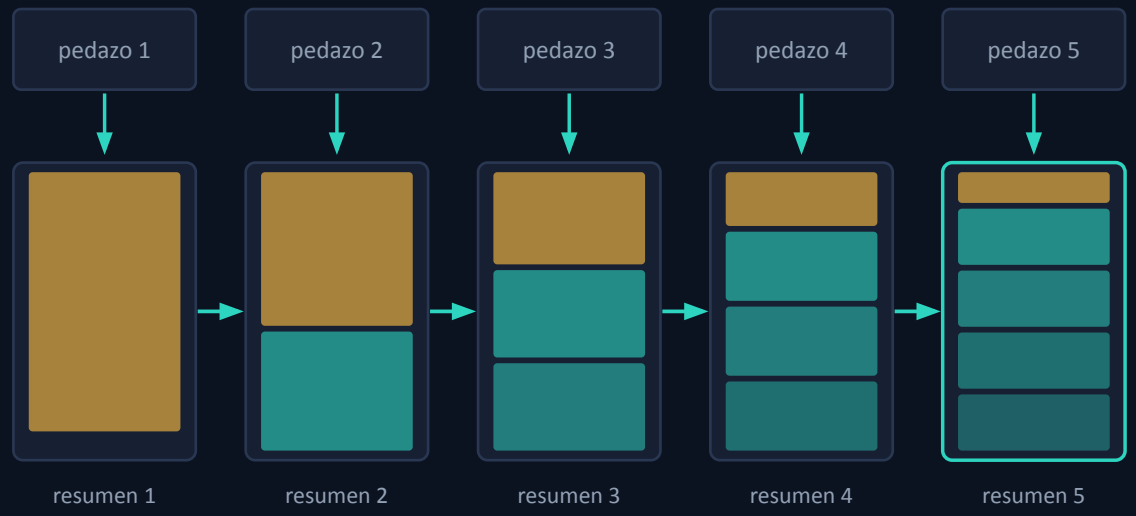
Map-reduce escala porque cada pedazo se resume solo, y por eso mismo pierde lo que vivía entre pedazos. ▸ Notebook §7 — el acto que no entraba en el embedder en la clase de RAG, resumido por las tres cadenas.

Refine: el resumen corrido, otra vez

Refine es el resumen corrido, aplicado a un documento. Se lee el primer pedazo y se resume; con el segundo, se actualiza el resumen; y así hasta el final: cada paso recibe el resumen anterior y el pedazo nuevo. Es lo mismo que hacía el resumen corrido con los turnos que salían de la ventana: una conversación es un documento que llega con el tiempo.

Su fortaleza es el contexto. A diferencia de map-reduce, cada paso sabe lo que vino antes, así que una relación entre pedazos puede sobrevivir: el resumen la puede arrastrar. En la prueba, con un modelo chico, no la arrastró.

Su costo es el orden. No se puede paralelizar: cada paso espera al anterior. Y lo del principio se recomprime en cada paso: el primer pedazo pasa por N resúmenes, el último por uno solo. Lo que entró temprano se va diluyendo. En la conversación pasó algo así: en el resumen de los turnos 0–12, el único número de reglamento que quedó es el del turno 12, el de los biocidas, el último que entró. Es el desgaste con el que cierra la semana.



ámbar, lo del pedazo 1: se recomprime N veces

cada tarjeta = f(la anterior, el pedazo nuevo)

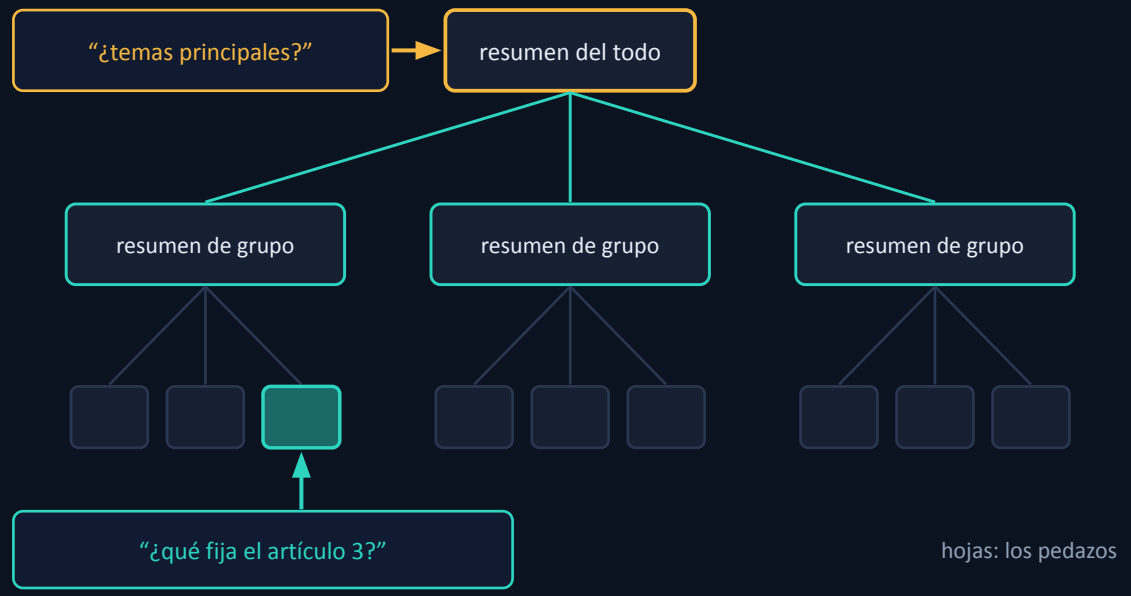
El resumen corrido de una conversación es un refine sobre un documento que llega de a pedazos: las mismas virtudes, y el mismo desgaste.

Resúmenes de resúmenes

Si un resumen de resúmenes funciona, se puede armar un árbol. OpenAI lo hizo para libros: secciones, resúmenes de secciones, resúmenes de esos resúmenes, hasta el libro entero. RAPTOR, de Stanford, dio el paso que lo conecta con RAG: agrupa los pedazos por parecido, resume cada grupo, agrupa los resúmenes, y repite. Y después busca en cualquier nivel del árbol: una pregunta global cae en un nodo alto; una de detalle, en una hoja. Así la pregunta global vuelve a tener pasaje: un resumen. Sobre preguntas de documentos largos, sumó 20 puntos de acierto.

GraphRAG, la semana que viene, hace algo parecido con un grafo: arma grupos de entidades relacionadas, resume cada grupo y contesta una pregunta global con map-reduce sobre esos resúmenes.

Es decidir al escribir. El árbol se arma una sola vez, antes de cualquier pregunta, y después contestar una global es una búsqueda más: tan rápida como cualquier turno. Map-reduce, en cambio, paga en cada pregunta. El árbol también tiene su precio: una llamada por nodo, que se paga entera al principio y otra vez cuando el corpus cambia; errores que se acumulan al subir; y cada resumen escrito antes de que existiera la pregunta.



la global cae arriba; la de detalle, en una hoja

Un árbol de resúmenes vuelve buscable lo global: cada nivel responde preguntas de distinto alcance. Ordenarlo por entidades y relaciones es la última parte de la clase.

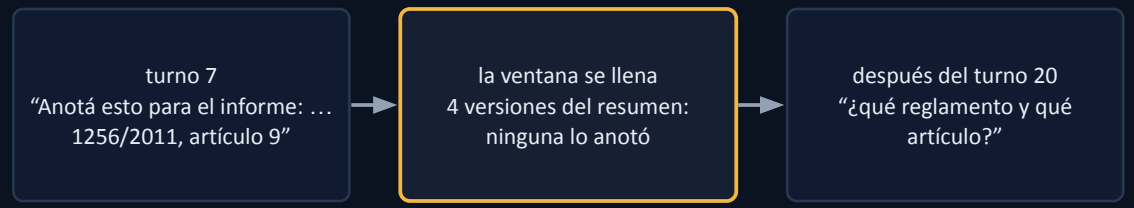
El resumen se escribió antes de la pregunta

Las políticas que olvidan la conversación tienen algo en común, y es estructural. Recortar y resumir deciden qué importa en el momento de sacar algo de la ventana, y en ese momento la pregunta que lo va a necesitar todavía no existe. Comprimir los pasajes no entra acá: trabaja sobre los pasajes de la pregunta de ahora.

Es la falla del vector precalculado. En RAG, el embedder calculaba el vector de cada pasaje antes de que existiera la pregunta, y tenía que adivinar qué era importante. Un resumen hace lo mismo con los turnos: si después se pregunta por un detalle que juzgó menor —un número, un artículo, lo primero que se pierde—, ya no está. En la prueba, ni un "anotá esto para el informe" lo salvó: ninguna de las cuatro versiones del resumen conservó el reglamento y el artículo.

Y es peor que en RAG. Allá, el pasaje original seguía en el corpus, y un re-puntuador podía volver a leerlo junto con la pregunta. Un resumen que reemplazó a los turnos no deja nada para releer. Salvo que se hayan descargado con un puntero: por eso descargar era la única política que no apostaba.

Es la parte "cuándo" de la pregunta madre. Resumir decide al escribir, sin la pregunta delante; descargar deja la decisión para cuando se lee, con la pregunta delante. Por eso, en la prueba, de las cuatro políticas sólo descargar devuelve el artículo 9.



la respuesta, según la política:

pegar todo	"…1256/2011, artículo 9" ✓
recortar · anclar · resumir	contesta "1382/2013", el del turno 20 ✗
descargar	vuelven los dos ✓

sin el dato, contestó con el del turno 20: un programa de justicia

Un resumen decide qué importa sin saber qué se va a preguntar. Es la falla del vector precalculado, sin el texto original para volver a leer. ▸ Notebook §8 — el identificador plantado, perdido en el resumen.

Cómo se equivoca un resumen

Un resumen no sólo pierde: también inventa y confunde. Lo midió un experimento publicado: Wang et al., el trabajo del resumen corrido, revisó a mano los resúmenes de cien diálogos contra lo que se había dicho. Encontró tres tipos de error, que juntos no llegan al 10% del contenido (los ejemplos son de la tabla del paper):

Inventados: el bot contó que camina al trabajo “y así ve animales”; el resumen dice que camina “para ver animales”, un motivo que nadie dijo.

Mal atribuidos: el usuario cedió ante sus hijas y les compró un gato; el resumen dice que cedió ante el gato.

Omitidos: el usuario nombró un documental, The Social Dilemma; el resumen dice que le gusta mirar televisión.

En nuestra prueba, el resumen sólo omitió; inventó el modelo, al contestar sin el dato.

Dos lecturas del mismo número. Los autores concluyen que los resúmenes son confiables. Para una memoria es mucho: el resumen se lee en cada turno como si fuera la conversación, y lo inventado pasa a ser algo que “se dijo”.

La auditoría es un método. Comparar el resumen con lo que pasó, contando esos tres errores, antes de confiarle una conversación: lo pide el segundo proyecto.



el experimento de Wang et al.: cien diálogos revisados a mano

Un resumen no sólo pierde: también inventa y confunde. Y lo que dice el resumen, el modelo lo lee como si se hubiera dicho.

Resumir el resumen

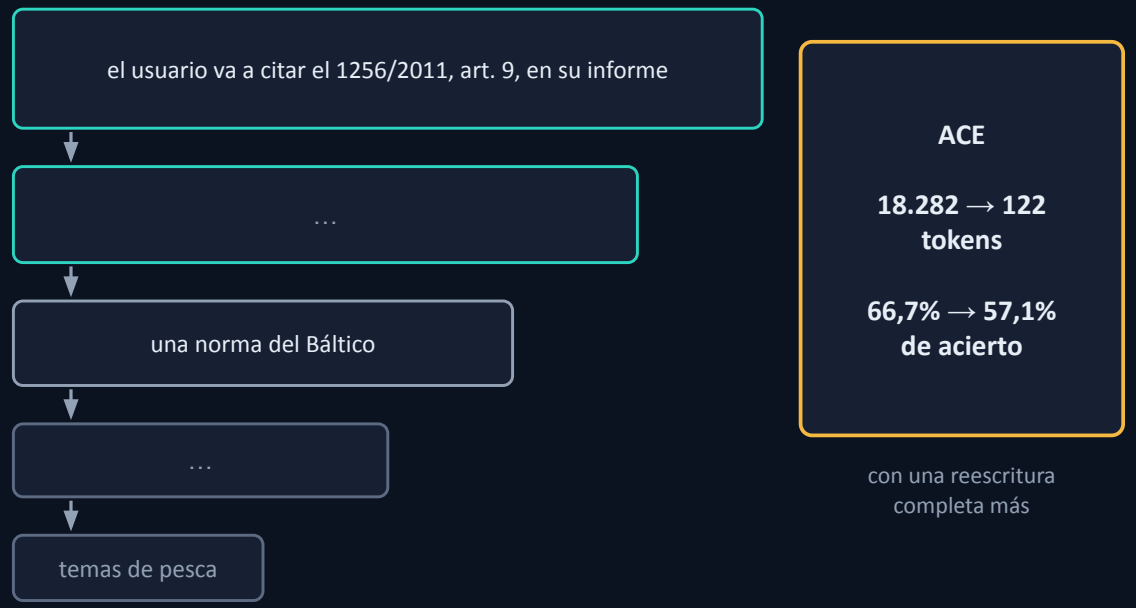
Cada re-resumen comprime otra vez lo ya comprimido. Un resumen corrido no se escribe una vez: se reescribe cada vez que el historial pasa su umbral. Y cada reescritura tira hacia un texto más corto y más genérico. En la prueba pasó en chico: después de cuatro reescrituras, el resumen de los turnos 0–12 era una lista de temas.

Dos efectos con nombre. Los nombra ACE (ICLR 2026), un trabajo sobre agentes que mantienen un manual de estrategias y lo van reescribiendo a medida que aprenden:

Sesgo de brevedad (brevity bias): la tendencia a colapsar hacia textos cortos y genéricos, que pierden los detalles del dominio.

Colapso de contexto (context collapse): lo que pasa cuando el modelo reescribe entero, en cada paso, todo lo acumulado. Su ejemplo es brutal: un manual de 18.282 tokens quedó en 122 después de una sola reescritura completa más, y el acierto cayó con él.

El antídoto: no reescribir, actualizar por partes. ACE trata el contexto como una lista de ítems y lo modifica con cambios chicos y localizados: se agrega, se edita o se borra un ítem, sin regenerar el resto. Es la misma idea con la que se mantiene al día la memoria de largo plazo, la semana que viene.



Reescribir un resumen una y otra vez lo gasta: cada vuelta es más corta y más genérica. Actualizar por partes en vez de reescribir entero es el antídoto.

ESTAMOS ACÁ · EL CIERRE DE LA SEMANA

Tres deudas con nombre

Lo que tenemos. Una conversación que funciona: el buscador recibe una pregunta completa, la ventana se arma en cada turno adentro de un presupuesto contado, el historial guarda lo que se dijo, y cuando no alcanza se olvida a propósito, con una apuesta explícita. En chico, es lo que el harness de Claude hace en cada mensaje.

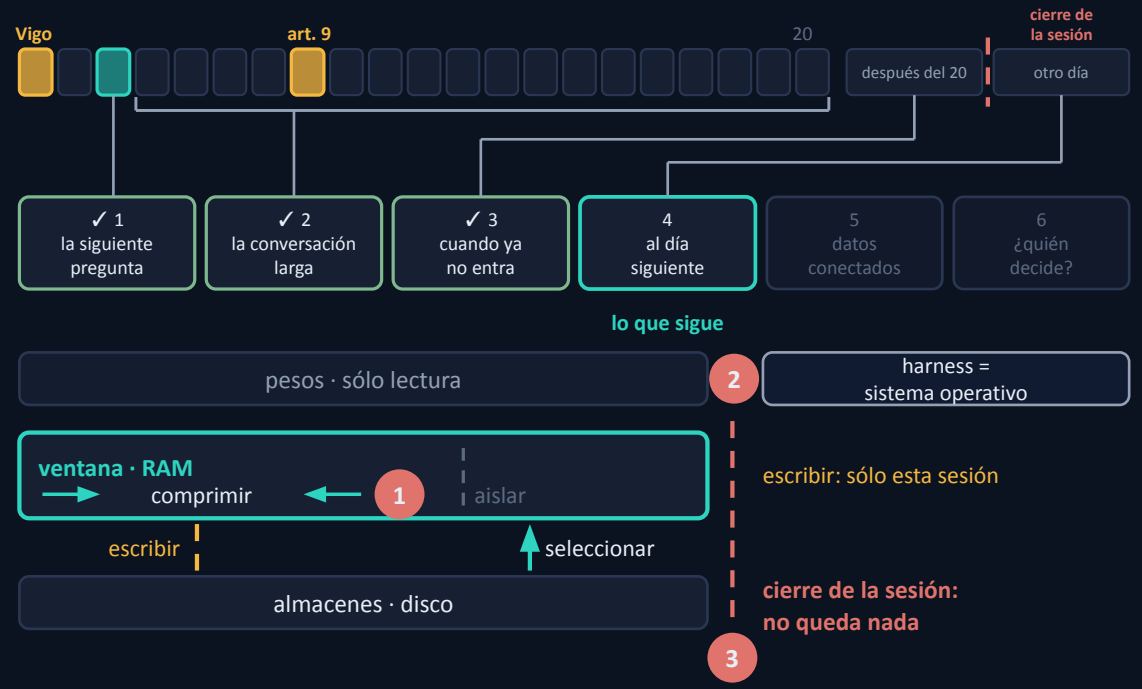
Tres deudas con nombre, no una sensación de que podría andar mejor:

Los resúmenes pierden e inventan, y deciden antes de la pregunta. Descargar con un puntero lo mitiga, sólo en parte.

Cada técnica cuesta llamadas y rompe caches. Ninguna es gratis: todas son compromisos.

Todo vive adentro de la sesión, y ésta es la que importa. Cuando la conversación se cierra, el historial, el resumen y lo descargado quedan en esa sesión, y la próxima empieza de cero.

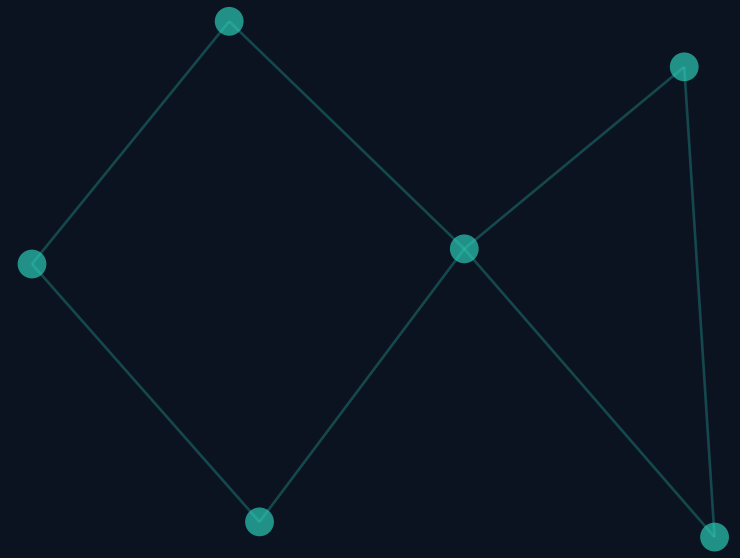
Como en Memento. Al protagonista la memoria se le borra a los pocos minutos, y lo único que sobrevive es lo que dejó escrito: notas, fotos, tatuajes. Claude, en un chat nuevo, se acuerda de lo que se le contó otro día. ¿Cómo, si el modelo no recuerda nada? Es la semana que viene: qué escribir afuera de la ventana, cómo mantenerlo al día, cómo volver a leerlo y cómo conectar lo escrito, en un grafo.



Esta semana termina con una conversación que funciona y tres deudas con nombre. La que queda abierta: cuando la sesión se cierra, no queda nada.

CLASE 9 · UNIDAD 4 · SEMANA 10

Memoria de largo plazo



Entre una sesión y la siguiente no sobrevive nada que no se haya escrito: qué se escribe, cómo se mantiene al día y cómo se vuelve a leer.